

CloudCart

Online Shopping Portal — React Frontend Source Code

Customer storefront + Admin console

Generated for internship project submission

Contents

1. package.json
2. .env.example
3. vite.config.js
4. tailwind.config.js
5. postcss.config.js
6. index.html
7. src/index.css
8. src/main.jsx
9. src/App.jsx
10. src/api/axiosClient.js
11. src/api/mockData.js
12. src/api/authApi.js
13. src/api/productApi.js
14. src/api/categoryApi.js
15. src/api/cartApi.js
16. src/api/orderApi.js
17. src/api/reviewApi.js
18. src/api/wishlistApi.js
19. src/api/adminApi.js
20. src/context/AuthContext.jsx
21. src/context/CartContext.jsx
22. src/components/common/ProtectedRoute.jsx
23. src/components/common/AdminRoute.jsx
24. src/components/common/Spinner.jsx
25. src/components/common/StarRating.jsx
26. src/components/common/Pagination.jsx
27. src/components/common/EmptyState.jsx
28. src/components/common/StatusBadge.jsx
29. src/components/layout/Navbar.jsx
30. src/components/layout/Footer.jsx
31. src/components/layout/StorefrontLayout.jsx
32. src/components/layout/AdminSidebar.jsx
33. src/components/layout/AdminTopbar.jsx
34. src/components/layout/AdminLayout.jsx
35. src/components/product/ProductImage.jsx
36. src/components/product/ProductCard.jsx
37. src/components/product/ProductGrid.jsx
38. src/components/product/CategoryFilter.jsx
39. src/components/product/ReviewList.jsx
40. src/components/product/ReviewForm.jsx
41. src/components/cart/CartItemRow.jsx
42. src/components/cart/CartSummary.jsx
43. src/pages/customer/Home.jsx
44. src/pages/customer/Login.jsx
45. src/pages/customer/Register.jsx
46. src/pages/customer/ForgotPassword.jsx
47. src/pages/customer/Products.jsx
48. src/pages/customer/ProductDetails.jsx
49. src/pages/customer/Wishlist.jsx
50. src/pages/customer/Cart.jsx
51. src/pages/customer/Checkout.jsx
52. src/pages/customer/Orders.jsx
53. src/pages/customer/OrderDetails.jsx
54. src/pages/customer/Profile.jsx
55. src/pages/customer/NotFound.jsx
56. src/pages/admin/AdminLogin.jsx
57. src/pages/admin/Dashboard.jsx
58. src/pages/admin/AdminProducts.jsx
59. src/pages/admin/AdminCategories.jsx
60. src/pages/admin/AdminInventory.jsx
61. src/pages/admin/AdminOrders.jsx
62. src/pages/admin/AdminCustomers.jsx
63. src/pages/admin/AdminReports.jsx
64. README.md

01 package.json

```
{
  "name": "cloudcart-frontend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "CloudCart - Online Shopping Portal (Customer + Admin React frontend)",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "eslint src --ext js,jsx"
  },
  "dependencies": {
    "axios": "^1.7.7",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.2"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "autoprefixer": "^10.4.20",
    "eslint": "^8.57.0",
    "eslint-plugin-react": "^7.35.0",
    "eslint-plugin-react-hooks": "^4.6.2",
    "postcss": "^8.4.47",
    "tailwindcss": "^3.4.13",
    "vite": "^5.4.8"
  }
}
```

02 .env.example

```
# Spring Boot backend base URL (see ../cloudcart-backend)
VITE_API_BASE_URL=http://localhost:8080/api

# Set to "false" once the backend above is running to switch off mock data
VITE_USE MOCK=true
```

03 vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// CloudCart frontend build configuration
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    open: true,
  },
  build: {
    outDir: "dist",
    sourcemap: false,
  },
});
```

```

/** CloudCart design tokens
 * Palette:
 *   ink      #14161F - primary text / dark surfaces
 *   cargo    #4338CA - primary brand (indigo) - "cloud" side of CloudCart
 *   cargo-d  #312A87 - primary hover/pressed
 *   crate    #FF6B4A - accent / CTA / sale badges - "cart" side, warm & alive
 *   mist     #F5F6FB - app background
 *   paper    #FFFFFF - card surfaces
 *   line     #E4E6F0 - hairline borders
 *   slate    #6B7080 - secondary text
 *   ok       #16A34A - success / in-stock
 *   warn     #D97706 - low stock / pending
 *   danger   #DC2626 - errors / out of stock
 */
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./*.html", "*/src/**/*.{js,jsx}"],
  theme: {
    extend: {
      colors: {
        ink: "#14161F",
        cargo: { DEFAULT: "#4338CA", dark: "#312A87", light: "#EEF0FD" },
        crate: { DEFAULT: "#FF6B4A", dark: "#E5502F", light: "#FFE9E1" },
        mist: "#F5F6FB",
        paper: "#FFFFFF",
        line: "#E4E6F0",
        slate: "#6B7080",
        ok: "#16A34A",
        warn: "#D97706",
        danger: "#DC2626",
      },
      fontFamily: {
        display: ["'Space Grotesk'", "sans-serif"],
        body: ["'Inter'", "sans-serif"],
        mono: ["'IBM Plex Mono'", "monospace"],
      },
      borderRadius: {
        card: "14px",
      },
      boxShadow: {
        crate: "0 1px 2px rgba(20,22,31,0.04), 0 8px 24px -8px rgba(67,56,202,0.18)",
        pop: "0 12px 32px -12px rgba(20,22,31,0.35)",
      },
      keyframes: {
        stackdrop: {
          "0%": { transform: "translateY(-6px) scale(0.85)", opacity: "0" },
          "60%": { transform: "translateY(2px) scale(1.05)", opacity: "1" },
          "100%": { transform: "translateY(0) scale(1)", opacity: "1" },
        },
      },
      animation: {
        stackdrop: "stackdrop 0.35s cubic-bezier(.2,.9,.3,1.2)",
      },
    },
  },
  plugins: [],
};

```

```
export default {  
  plugins: {  
    tailwindcss: {},  
    autoprefixer: {},  
  },  
};
```

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>CloudCart – Shop smarter, shipped faster</title>
    <meta name="description" content="CloudCart: a cloud-native online shopping portal." />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@500;600;700&family=Inter:wght@400;500;600;700&family=IBM+Plex+Mono:wght@500;600&display=swap"
      rel="stylesheet"
    />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

```

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html {
    @apply scroll-smooth;
  }
  body {
    @apply bg-mist text-ink font-body antialiased;
  }
  h1, h2, h3, h4 {
    @apply font-display tracking-tight;
  }
  /* Visible keyboard focus everywhere, per accessibility floor */
  a:focus-visible,
  button:focus-visible,
  input:focus-visible,
  select:focus-visible,
  textarea:focus-visible {
    @apply outline outline-2 outline-offset-2 outline-cargo;
  }
  @media (prefers-reduced-motion: reduce) {
    * {
      animation-duration: 0.01ms !important;
      animation-iteration-count: 1 !important;
      transition-duration: 0.01ms !important;
    }
  }
}

@layer components {
  .btn {
    @apply inline-flex items-center justify-center gap-2 rounded-full px-5 py-2.5 text-sm font-semibold
      transition-colors duration-150 disabled:opacity-50 disabled:cursor-not-allowed;
  }
  .btn-primary {
    @apply btn bg-cargo text-white hover:bg-cargo-dark;
  }
  .btn-accent {
    @apply btn bg-crate text-white hover:bg-crate-dark;
  }
  .btn-ghost {
    @apply btn bg-transparent text-ink border border-line hover:bg-mist;
  }
  .btn-danger {
    @apply btn bg-danger text-white hover:bg-red-700;
  }
  .card {
    @apply bg-paper rounded-card border border-line shadow-crate;
  }
  .input {
    @apply w-full rounded-lg border border-line bg-paper px-3.5 py-2.5 text-sm text-ink
      placeholder:text-slate/70 focus:border-cargo outline-none transition-colors;
  }
  .label {
    @apply mb-1.5 block text-sm font-medium text-ink;
  }
  .eyebrow {
    @apply text-xs font-mono font-semibold uppercase tracking-widest text-cargo;
  }
  .price {
    @apply font-mono font-semibold text-ink;
  }
  .badge {
    @apply inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-semibold font-mono;
  }
  .badge-ok {
    @apply badge bg-ok/10 text-ok;
  }
  .badge-warn {
    @apply badge bg-warn/10 text-warn;
  }
  .badge-danger {
    @apply badge bg-danger/10 text-danger;
  }
}

```

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

```

import { BrowserRouter, Routes, Route } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import { CartProvider } from "../context/CartContext";

import StorefrontLayout from "../components/layout/StorefrontLayout";
import AdminLayout from "../components/layout/AdminLayout";
import ProtectedRoute from "../components/common/ProtectedRoute";
import AdminRoute from "../components/common/AdminRoute";

import Home from "../pages/customer/Home";
import Login from "../pages/customer/Login";
import Register from "../pages/customer/Register";
import ForgotPassword from "../pages/customer/ForgotPassword";
import Products from "../pages/customer/Products";
import ProductDetails from "../pages/customer/ProductDetails";
import Wishlist from "../pages/customer/Wishlist";
import Cart from "../pages/customer/Cart";
import Checkout from "../pages/customer/Checkout";
import Orders from "../pages/customer/Orders";
import OrderDetails from "../pages/customer/OrderDetails";
import Profile from "../pages/customer/Profile";
import NotFound from "../pages/customer/NotFound";

import AdminLogin from "../pages/admin/AdminLogin";
import Dashboard from "../pages/admin/Dashboard";
import AdminProducts from "../pages/admin/AdminProducts";
import AdminCategories from "../pages/admin/AdminCategories";
import AdminInventory from "../pages/admin/AdminInventory";
import AdminOrders from "../pages/admin/AdminOrders";
import AdminCustomers from "../pages/admin/AdminCustomers";
import AdminReports from "../pages/admin/AdminReports";

export default function App() {
  return (
    <AuthProvider>
      <CartProvider>
        <BrowserRouter>
          <Routes>
            { /* Admin login sits outside both shells */ }
            <Route path="/admin/login" element={ <AdminLogin /> } />

            { /* Admin area */ }
            <Route
              path="/admin"
              element={
                <AdminRoute>
                  <AdminLayout />
                </AdminRoute>
              }
            />

            <Route path="/dashboard" element={ <Dashboard /> } />
            <Route path="/products" element={ <AdminProducts /> } />
            <Route path="/categories" element={ <AdminCategories /> } />
            <Route path="/inventory" element={ <AdminInventory /> } />
            <Route path="/orders" element={ <AdminOrders /> } />
            <Route path="/customers" element={ <AdminCustomers /> } />
            <Route path="/reports" element={ <AdminReports /> } />
          </Routes>

          { /* Customer storefront */ }
          <Route path="/" element={ <StorefrontLayout /> }>
            <Route index element={ <Home /> } />
            <Route path="/login" element={ <Login /> } />
            <Route path="/register" element={ <Register /> } />
            <Route path="/forgot-password" element={ <ForgotPassword /> } />
            <Route path="/products" element={ <Products /> } />
            <Route path="/products/:id" element={ <ProductDetails /> } />
            <Route path="/wishlist" element={ <Wishlist /> } />
            <Route path="/cart" element={ <Cart /> } />
            <Route
              path="/checkout"
              element={
                <ProtectedRoute>
                  <Checkout />
                </ProtectedRoute>
              }
            />
            <Route
              path="/orders"
              element={
                <ProtectedRoute>
                  <Orders />
                </ProtectedRoute>
              }
            />
            <Route
              path="/orders/:id"
              element={
                <ProtectedRoute>
                  <OrderDetails />
                </ProtectedRoute>
              }
            />
            <Route
              path="/profile"
              element={
                <ProtectedRoute>
                  <Profile />
                </ProtectedRoute>
              }
            />
            <Route path="*" element={ <NotFound /> } />
          </Route>
        </BrowserRouter>
      </CartProvider>
    </AuthProvider>
  );
}

```

```
</AuthProvider>  
);  
}
```

```

import axios from "axios";

// Base URL points at the Spring Boot backend. Configure via .env (see .env.example).
const BASE_URL = import.meta.env.VITE_API_BASE_URL || "http://localhost:8080/api";

const axiosClient = axios.create({
  baseURL: BASE_URL,
  headers: { "Content-Type": "application/json" },
});

// Attach the access token to every outgoing request.
axiosClient.interceptors.request.use((config) => {
  const token = localStorage.getItem("cloudcart_access_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// On 401, try once to refresh the access token using the stored refresh token.
let isRefreshing = false;
let pendingQueue = [];

const processQueue = (error, token = null) => {
  pendingQueue.forEach(({ resolve, reject }) => {
    if (error) reject(error);
    else resolve(token);
  });
  pendingQueue = [];
};

axiosClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;
    if (error.response?.status === 401 && !originalRequest._retry) {
      if (isRefreshing) {
        return new Promise((resolve, reject) => {
          pendingQueue.push({ resolve, reject });
        }).then((token) => {
          originalRequest.headers.Authorization = `Bearer ${token}`;
          return axiosClient(originalRequest);
        });
      }
      originalRequest._retry = true;
      isRefreshing = true;
      const refreshToken = localStorage.getItem("cloudcart_refresh_token");

      try {
        const { data } = await axios.post(`${BASE_URL}/auth/refresh-token`, { refreshToken });
        localStorage.setItem("cloudcart_access_token", data.accessToken);
        processQueue(null, data.accessToken);
        originalRequest.headers.Authorization = `Bearer ${data.accessToken}`;
        return axiosClient(originalRequest);
      } catch (refreshError) {
        processQueue(refreshError, null);
        localStorage.removeItem("cloudcart_access_token");
        localStorage.removeItem("cloudcart_refresh_token");
        localStorage.removeItem("cloudcart_user");
        window.location.href = "/login";
        return Promise.reject(refreshError);
      } finally {
        isRefreshing = false;
      }
    }
    return Promise.reject(error);
  }
);

export default axiosClient;

```

11 src/api/mockData.js

```
// CloudCart ships wired to real REST endpoints (see each api/*.js file).
// Until the Spring Boot backend is running, VITE_USE MOCK=true serves this
// in-memory data instead, so the UI can be reviewed standalone.
export const USE MOCK = import.meta.env.VITE_USE MOCK !== "false";

export const mockCategories = [
  { id: 1, name: "Laptops & Computers", slug: "laptops-computers" },
  { id: 2, name: "Audio & Headphones", slug: "audio-headphones" },
  { id: 3, name: "Smart Home", slug: "smart-home" },
  { id: 4, name: "Wearables", slug: "wearables" },
  { id: 5, name: "Gaming", slug: "gaming" },
  { id: 6, name: "Cameras", slug: "cameras" },
];

export const mockProducts = [
  { id: 101, name: "Aria 14\" Ultrabook", categoryId: 1, price: 1249.0, stock: 18, rating: 4.6, reviewCount: 132, image: "laptop-ultrabook", description: "A featherlight 14-inch ultrabook with a 32-hour battery and a 3K display, built for people who work from anywhere." },
  { id: 102, name: "Forge 16 Creator Laptop", categoryId: 1, price: 1899.0, stock: 7, rating: 4.8, reviewCount: 89, image: "laptop-creator", description: "A 16-inch workstation-class laptop with discrete graphics for video editing and 3D rendering on the move." },
  { id: 103, name: "Nimbus Wireless ANC Headphones", categoryId: 2, price: 279.0, stock: 42, rating: 4.5, reviewCount: 401, image: "headphones-anc", description: "Adaptive noise cancelling headphones tuned for long commutes, with 40 hours of playback per charge." },
  { id: 104, name: "Pebble True Wireless Earbuds", categoryId: 2, price: 129.0, stock: 65, rating: 4.3, reviewCount: 268, image: "earbuds", description: "Compact true wireless earbuds with a pocketable case and IPX5 sweat resistance." },
  { id: 105, name: "Hearth Smart Thermostat", categoryId: 3, price: 189.0, stock: 30, rating: 4.4, reviewCount: 156, image: "thermostat", description: "Learns your schedule and trims energy bills automatically, with a companion app for every room." },
  { id: 106, name: "Halo Smart Bulb 4-Pack", categoryId: 3, price: 59.0, stock: 120, rating: 4.2, reviewCount: 512, image: "smart-bulb", description: "Tunable white and color smart bulbs that pair directly with CloudCart's Smart Home app." },
  { id: 107, name: "Pulse Fitness Watch", categoryId: 4, price: 219.0, stock: 24, rating: 4.6, reviewCount: 233, image: "smartwatch", description: "Tracks heart rate, sleep, and 40+ workout types with a battery that lasts a full week." },
  { id: 108, name: "Drift Ring Tracker", categoryId: 4, price: 299.0, stock: 15, rating: 4.1, reviewCount: 74, image: "smart-ring", description: "A titanium sleep and recovery tracker worn on the finger, with a 6-day battery." },
  { id: 109, name: "Vertex Mechanical Keyboard", categoryId: 5, price: 149.0, stock: 38, rating: 4.7, reviewCount: 310, image: "keyboard", description: "Hot-swappable mechanical keyboard with per-key RGB and a machined aluminum frame." },
  { id: 110, name: "Ranger Gaming Mouse", categoryId: 5, price: 79.0, stock: 54, rating: 4.5, reviewCount: 198, image: "mouse", description: "A 62-gram wireless gaming mouse with a 26k DPI sensor and zero click lag." },
  { id: 111, name: "Aperture Mirrorless Camera", categoryId: 6, price: 1099.0, stock: 9, rating: 4.8, reviewCount: 61, image: "camera", description: "A 26MP mirrorless camera with in-body stabilization, built for creators who shoot on the go." },
  { id: 112, name: "Lumen Vlogging Kit", categoryId: 6, price: 249.0, stock: 22, rating: 4.4, reviewCount: 118, image: "vlog-kit", description: "A compact camera, mic, and mini-tripod kit designed for one-person video production." },
];

export const mockReviews = [
  { id: 1, productId: 101, userName: "Meera K.", rating: 5, comment: "Battery genuinely lasts the full day of meetings.", date: "2026-05-02" },
  { id: 2, productId: 101, userName: "Daniel O.", rating: 4, comment: "Fast and light, wish the webcam was better.", date: "2026-04-18" },
  { id: 3, productId: 103, userName: "Priya S.", rating: 5, comment: "Best ANC I've used at this price point.", date: "2026-06-01" },
];

export const mockUsers = [
  { id: 1, name: "Admin User", email: "admin@cloudcart.com", password: "Admin@123", role: "ADMIN" },
  { id: 2, name: "Asha Rao", email: "asha@example.com", password: "Password@123", role: "CUSTOMER" },
];

export const mockOrders = [
  {
    id: 5001,
    userId: 2,
    status: "DELIVERED",
    placedAt: "2026-06-20",
    total: 1408.0,
    items: [
      { productId: 101, name: "Aria 14\" Ultrabook", quantity: 1, price: 1249.0 },
      { productId: 104, name: "Pebble True Wireless Earbuds", quantity: 1, price: 129.0 },
    ],
  },
  {
    id: 5002,
    userId: 2,
    status: "SHIPPED",
    placedAt: "2026-07-02",
    total: 279.0,
    items: [{ productId: 103, name: "Nimbus Wireless ANC Headphones", quantity: 1, price: 279.0 }],
  },
];

export const mockCustomers = [
  { id: 2, name: "Asha Rao", email: "asha@example.com", orders: 2, totalSpent: 1687.0, joined: "2025-11-04" },
  { id: 3, name: "Vikram Nair", email: "vikram@example.com", orders: 5, totalSpent: 3421.0, joined: "2025-08-19" },
  { id: 4, name: "Lena Chu", email: "lena@example.com", orders: 1, totalSpent: 189.0, joined: "2026-03-10" },
];
```

12 src/api/authApi.js

```
import axiosClient from "../axiosClient";
import { USE_MOCK, mockUsers } from "../mockData";

const delay = (ms = 350) => new Promise((res) => setTimeout(res, ms));

// POST /api/auth/register
export async function register({ name, email, password }) {
  if (USE_MOCK) {
    await delay();
    if (mockUsers.some((u) => u.email === email)) {
      throw { response: { data: { message: "An account with this email already exists." } } };
    }
    const user = { id: mockUsers.length + 1, name, email, password, role: "CUSTOMER" };
    mockUsers.push(user);
    return { message: "Registration successful. Please log in." };
  }
  const { data } = await axiosClient.post("/auth/register", { name, email, password });
  return data;
}

// POST /api/auth/login
export async function login({ email, password }) {
  if (USE_MOCK) {
    await delay();
    const user = mockUsers.find((u) => u.email === email && u.password === password);
    if (!user) throw { response: { data: { message: "Invalid email or password." } } };
    return {
      accessToken: `mock-access-${user.id}`,
      refreshToken: `mock-refresh-${user.id}`,
      user: { id: user.id, name: user.name, email: user.email, role: user.role },
    };
  }
  const { data } = await axiosClient.post("/auth/login", { email, password });
  return data;
}

// POST /api/auth/logout
export async function logout() {
  if (USE_MOCK) {
    await delay(150);
    return { message: "Logged out." };
  }
  const { data } = await axiosClient.post("/auth/logout");
  return data;
}

// POST /api/auth/refresh-token
export async function refreshToken(token) {
  if (USE_MOCK) {
    await delay(150);
    return { accessToken: `mock-access-refreshed` };
  }
  const { data } = await axiosClient.post("/auth/refresh-token", { refreshToken: token });
  return data;
}

// POST /api/auth/forgot-password
export async function forgotPassword(email) {
  if (USE_MOCK) {
    await delay();
    return { message: `If an account exists for ${email}, a reset link has been sent.` };
  }
  const { data } = await axiosClient.post("/auth/forgot-password", { email });
  return data;
}
```

13 src/api/productApi.js

```
import axiosClient from "../axiosClient";
import { USE_MOCK, mockProducts } from "../mockData";

const delay = (ms = 300) => new Promise((res) => setTimeout(res, ms));
let localProducts = [...mockProducts];

// GET /api/products?search=&categoryId=&page=&size=
export async function getProducts({ search = "", categoryId = null, page = 1, size = 12 } = {}) {
  if (USE_MOCK) {
    await delay();
    let results = [...localProducts];
    if (search) {
      const q = search.toLowerCase();
      results = results.filter((p) => p.name.toLowerCase().includes(q) || p.description.toLowerCase().includes(q));
    }
    if (categoryId) results = results.filter((p) => p.categoryId === Number(categoryId));
    const start = (page - 1) * size;
    return {
      content: results.slice(start, start + size),
      totalElements: results.length,
      totalPages: Math.max(1, Math.ceil(results.length / size)),
      page,
    };
  }
  const { data } = await axiosClient.get("/products", { params: { search, categoryId, page, size } });
  return data;
}

// GET /api/products/{id}
export async function getProductById(id) {
  if (USE_MOCK) {
    await delay();
    const product = localProducts.find((p) => p.id === Number(id));
    if (!product) throw { response: { data: { message: "Product not found." } } };
    return product;
  }
  const { data } = await axiosClient.get(`/products/${id}`);
  return data;
}

// POST /api/products (admin only)
export async function createProduct(product) {
  if (USE_MOCK) {
    await delay();
    const newProduct = { ...product, id: Math.max(...localProducts.map((p) => p.id)) + 1, rating: 0, reviewCount: 0 };
    localProducts.push(newProduct);
    return newProduct;
  }
  const { data } = await axiosClient.post("/products", product);
  return data;
}

// PUT /api/products/{id} (admin only)
export async function updateProduct(id, updates) {
  if (USE_MOCK) {
    await delay();
    localProducts = localProducts.map((p) => (p.id === Number(id) ? { ...p, ...updates } : p));
    return localProducts.find((p) => p.id === Number(id));
  }
  const { data } = await axiosClient.put(`/products/${id}`, updates);
  return data;
}

// DELETE /api/products/{id} (admin only)
export async function deleteProduct(id) {
  if (USE_MOCK) {
    await delay();
    localProducts = localProducts.filter((p) => p.id !== Number(id));
    return { message: "Product deleted." };
  }
  const { data } = await axiosClient.delete(`/products/${id}`);
  return data;
}
```

14 src/api/categoryApi.js

```
import axiosClient from "../axiosClient";
import { USE_MOCK, mockCategories } from "../mockData";

const delay = (ms = 250) => new Promise((res) => setTimeout(res, ms));
let localCategories = [...mockCategories];

// GET /api/categories
export async function getCategories() {
  if (USE_MOCK) {
    await delay();
    return localCategories;
  }
  const { data } = await axiosClient.get("/categories");
  return data;
}

// POST /api/categories (admin only)
export async function createCategory(category) {
  if (USE_MOCK) {
    await delay();
    const newCategory = { ...category, id: Math.max(...localCategories.map((c) => c.id)) + 1 };
    localCategories.push(newCategory);
    return newCategory;
  }
  const { data } = await axiosClient.post("/categories", category);
  return data;
}

// PUT /api/categories/{id} (admin only)
export async function updateCategory(id, updates) {
  if (USE_MOCK) {
    await delay();
    localCategories = localCategories.map((c) => (c.id === Number(id) ? { ...c, ...updates } : c));
    return localCategories.find((c) => c.id === Number(id));
  }
  const { data } = await axiosClient.put(`/categories/${id}`, updates);
  return data;
}

// DELETE /api/categories/{id} (admin only)
export async function deleteCategory(id) {
  if (USE_MOCK) {
    await delay();
    localCategories = localCategories.filter((c) => c.id !== Number(id));
    return { message: "Category deleted." };
  }
  const { data } = await axiosClient.delete(`/categories/${id}`);
  return data;
}
```

```

import axiosClient from "../axiosClient";
import { USE_MOCK, mockProducts } from "../mockData";

const delay = (ms = 200) => new Promise((res) => setTimeout(res, ms));
const STORAGE_KEY = "cloudcart_cart_items"; // [{ productId, quantity }]

function readLocalCart() {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  } catch {
    return [];
  }
}

function writeLocalCart(items) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
}

function hydrate(items) {
  return items
    .map(({ productId, quantity }) => {
      const product = mockProducts.find((p) => p.id === productId);
      if (!product) return null;
      return { productId, quantity, name: product.name, price: product.price, image: product.image, stock: product.stock };
    })
    .filter(Boolean);
}

// GET /api/cart
export async function getCart() {
  if (USE_MOCK) {
    await delay();
    const items = hydrate(readLocalCart());
    const subtotal = items.reduce((sum, i) => sum + i.price * i.quantity, 0);
    return { items, subtotal };
  }
  const { data } = await axiosClient.get("/cart");
  return data;
}

// POST /api/cart/items
export async function addCartItem(productId, quantity = 1) {
  if (USE_MOCK) {
    await delay();
    const items = readLocalCart();
    const existing = items.find((i) => i.productId === productId);
    if (existing) existing.quantity += quantity;
    else items.push({ productId, quantity });
    writeLocalCart(items);
    return { items: hydrate(items) };
  }
  const { data } = await axiosClient.post("/cart/items", { productId, quantity });
  return data;
}

// PUT /api/cart/items/{productId}
export async function updateCartItem(productId, quantity) {
  if (USE_MOCK) {
    await delay();
    let items = readLocalCart();
    items = items.map((i) => (i.productId === productId ? { ...i, quantity } : i));
    writeLocalCart(items);
    return { items: hydrate(items) };
  }
  const { data } = await axiosClient.put(`/cart/items/${productId}`, { quantity });
  return data;
}

// DELETE /api/cart/items/{productId}
export async function removeCartItem(productId) {
  if (USE_MOCK) {
    await delay();
    const items = readLocalCart().filter((i) => i.productId !== productId);
    writeLocalCart(items);
    return { items: hydrate(items) };
  }
  const { data } = await axiosClient.delete(`/cart/items/${productId}`);
  return data;
}

// Clear cart after successful checkout (mock convenience helper)
export async function clearCart() {
  if (USE_MOCK) {
    await delay(100);
    writeLocalCart([]);
    return { items: [] };
  }
  const { data } = await axiosClient.delete("/cart");
  return data;
}

```

```

import axiosClient from "../axiosClient";
import { USE_MOCK, mockOrders } from "../mockData";

const delay = (ms = 350) => new Promise((res) => setTimeout(res, ms));
let localOrders = [...mockOrders];

// POST /api/orders
export async function placeOrder({ items, shippingAddress, paymentMethod }) {
  if (USE_MOCK) {
    await delay(600);
    const total = items.reduce((sum, i) => sum + i.price * i.quantity, 0);
    const order = {
      id: 5000 + localOrders.length + 1,
      userId: 2,
      status: "PLACED",
      placedAt: new Date().toISOString().slice(0, 10),
      total,
      items,
      shippingAddress,
      paymentMethod,
    };
    localOrders.push(order);
    return order;
  }
  const { data } = await axiosClient.post("/orders", { items, shippingAddress, paymentMethod });
  return data;
}

// GET /api/orders
export async function getOrders() {
  if (USE_MOCK) {
    await delay();
    return localOrders;
  }
  const { data } = await axiosClient.get("/orders");
  return data;
}

// GET /api/orders/{id}
export async function getOrderByid(id) {
  if (USE_MOCK) {
    await delay();
    const order = localOrders.find((o) => o.id === Number(id));
    if (!order) throw { response: { data: { message: "Order not found." } } };
    return order;
  }
  const { data } = await axiosClient.get(`/orders/${id}`);
  return data;
}

// PUT /api/orders/{id}/cancel
export async function cancelOrder(id) {
  if (USE_MOCK) {
    await delay();
    localOrders = localOrders.map((o) => (o.id === Number(id) ? { ...o, status: "CANCELLED" } : o));
    return localOrders.find((o) => o.id === Number(id));
  }
  const { data } = await axiosClient.put(`/orders/${id}/cancel`);
  return data;
}

// GET /api/orders/{id}/invoice (returns a PDF blob from the backend)
export async function downloadInvoice(id) {
  if (USE_MOCK) {
    await delay();
    return { message: "Invoice download is available once connected to the live backend." };
  }
  const { data } = await axiosClient.get(`/orders/${id}/invoice`, { responseType: "blob" });
  return data;
}

```

17 src/api/reviewApi.js

```
import axiosClient from "../axiosClient";
import { USE MOCK, mockReviews } from "../mockData";

const delay = (ms = 250) => new Promise((res) => setTimeout(res, ms));
let localReviews = [...mockReviews];

// GET /api/products/{productId}/reviews
export async function getReviews(productId) {
  if (USE MOCK) {
    await delay();
    return localReviews.filter((r) => r.productId === Number(productId));
  }
  const { data } = await axiosClient.get(`/products/${productId}/reviews`);
  return data;
}

// POST /api/products/{productId}/reviews
export async function addReview(productId, { rating, comment, userName }) {
  if (USE MOCK) {
    await delay();
    const review = {
      id: localReviews.length + 1,
      productId: Number(productId),
      rating,
      comment,
      userName: userName || "You",
      date: new Date().toISOString().slice(0, 10),
    };
    localReviews.push(review);
    return review;
  }
  const { data } = await axiosClient.post(`/products/${productId}/reviews`, { rating, comment });
  return data;
}
```

```

import axiosClient from "../axiosClient";
import { mockProducts, USE MOCK } from "../mockData";

const delay = (ms = 200) => new Promise((res) => setTimeout(res, ms));
const STORAGE_KEY = "cloudcart_wishlist"; // [productId]

function readLocal() {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  } catch {
    return [];
  }
}

function writeLocal(ids) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(ids));
}

// GET /api/wishlist
export async function getWishlist() {
  if (USE MOCK) {
    await delay();
    const ids = readLocal();
    return mockProducts.filter((p) => ids.includes(p.id));
  }
  const { data } = await axiosClient.get("/wishlist");
  return data;
}

// POST /api/wishlist/{productId}
export async function addToWishlist(productId) {
  if (USE MOCK) {
    await delay();
    const ids = new Set(readLocal());
    ids.add(productId);
    writeLocal([...ids]);
    return { message: "Added to wishlist." };
  }
  const { data } = await axiosClient.post(`/wishlist/${productId}`);
  return data;
}

// DELETE /api/wishlist/{productId}
export async function removeFromWishlist(productId) {
  if (USE MOCK) {
    await delay();
    writeLocal(readLocal().filter((id) => id !== productId));
    return { message: "Removed from wishlist." };
  }
  const { data } = await axiosClient.delete(`/wishlist/${productId}`);
  return data;
}

```

```

import axiosClient from "../axiosClient";
import { USE_MOCK, mockOrders, mockProducts, mockCustomers } from "../mockData";

const delay = (ms = 300) => new Promise((res) => setTimeout(res, ms));

// GET /api/admin/dashboard
export async function getDashboardStats() {
  if (USE_MOCK) {
    await delay();
    const revenue = mockOrders.reduce((sum, o) => sum + o.total, 0);
    return {
      totalRevenue: revenue,
      totalOrders: mockOrders.length,
      totalCustomers: mockCustomers.length,
      totalProducts: mockProducts.length,
      lowStockCount: mockProducts.filter((p) => p.stock < 10).length,
      recentOrders: mockOrders.slice(-5).reverse(),
      revenueByMonth: [
        { month: "Feb", revenue: 8200 },
        { month: "Mar", revenue: 10400 },
        { month: "Apr", revenue: 9100 },
        { month: "May", revenue: 13850 },
        { month: "Jun", revenue: 15720 },
        { month: "Jul", revenue: 9600 },
      ],
    };
  }
  const { data } = await axiosClient.get("/admin/dashboard");
  return data;
}

// GET /api/admin/reports/sales
export async function getSalesReport({ from, to } = {}) {
  if (USE_MOCK) {
    await delay();
    return {
      from: from || "2026-01-01",
      to: to || "2026-07-09",
      totalRevenue: mockOrders.reduce((sum, o) => sum + o.total, 0),
      totalOrders: mockOrders.length,
      averageOrderValue: mockOrders.reduce((sum, o) => sum + o.total, 0) / (mockOrders.length || 1),
      topProducts: [
        { name: "Aria 14\" Ultrabook", unitsSold: 41, revenue: 51209 },
        { name: "Nimbus Wireless ANC Headphones", unitsSold: 76, revenue: 21204 },
        { name: "Vertex Mechanical Keyboard", unitsSold: 58, revenue: 8642 },
      ],
    };
  }
  const { data } = await axiosClient.get("/admin/reports/sales", { params: { from, to } });
  return data;
}

// GET /api/admin/customers
export async function getCustomers() {
  if (USE_MOCK) {
    await delay();
    return mockCustomers;
  }
  const { data } = await axiosClient.get("/admin/customers");
  return data;
}

// PUT /api/admin/customers/{id}/status
export async function setCustomerStatus(id, status) {
  if (USE_MOCK) {
    await delay();
    return { id, status, message: `Customer ${status === "BLOCKED" ? "blocked" : "activated"}` };
  }
  const { data } = await axiosClient.put(`/admin/customers/${id}/status`, { status });
  return data;
}

// GET /api/admin/orders (all orders, any customer)
export async function getAllOrders() {
  if (USE_MOCK) {
    await delay();
    return mockOrders;
  }
  const { data } = await axiosClient.get("/admin/orders");
  return data;
}

// PUT /api/admin/orders/{id}/status
export async function updateOrderStatus(id, status) {
  if (USE_MOCK) {
    await delay();
    const order = mockOrders.find((o) => o.id === Number(id));
    if (order) order.status = status;
    return order;
  }
  const { data } = await axiosClient.put(`/admin/orders/${id}/status`, { status });
  return data;
}

```

```

import { createContext, useContext, useState, useCallback } from "react";
import * as authApi from "../api/authApi";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    try {
      return JSON.parse(localStorage.getItem("cloudcart_user")) || null;
    } catch {
      return null;
    }
  });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const persistSession = (data) => {
    localStorage.setItem("cloudcart_access_token", data.accessToken);
    localStorage.setItem("cloudcart_refresh_token", data.refreshToken);
    localStorage.setItem("cloudcart_user", JSON.stringify(data.user));
    setUser(data.user);
  };

  const login = useCallback(async (credentials) => {
    setLoading(true);
    setError(null);
    try {
      const data = await authApi.login(credentials);
      persistSession(data);
      return data.user;
    } catch (err) {
      const message = err?.response?.data?.message || "Unable to log in. Please try again.";
      setError(message);
      throw new Error(message);
    } finally {
      setLoading(false);
    }
  }, []);

  const register = useCallback(async (payload) => {
    setLoading(true);
    setError(null);
    try {
      return await authApi.register(payload);
    } catch (err) {
      const message = err?.response?.data?.message || "Registration failed. Please try again.";
      setError(message);
      throw new Error(message);
    } finally {
      setLoading(false);
    }
  }, []);

  const logout = useCallback(async () => {
    try {
      await authApi.logout();
    } finally {
      localStorage.removeItem("cloudcart_access_token");
      localStorage.removeItem("cloudcart_refresh_token");
      localStorage.removeItem("cloudcart_user");
      setUser(null);
    }
  }, []);

  const value = {
    user,
    isAuthenticated: !!user,
    isAdmin: user?.role === "ADMIN",
    loading,
    error,
    login,
    register,
    logout,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within an AuthProvider");
  return ctx;
}

```

21 src/context/CartContext.jsx

```
import { createContext, useContext, useState, useCallback, useEffect } from "react";
import * as cartApi from "../api/cartApi";

const CartContext = createContext(null);

export function CartProvider({ children }) {
  const [items, setItems] = useState([]);
  const [subtotal, setSubtotal] = useState(0);
  const [loading, setLoading] = useState(false);

  const refreshCart = useCallback(async () => {
    setLoading(true);
    try {
      const data = await cartApi.getCart();
      setItems(data.items);
      setSubtotal(data.subtotal);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    refreshCart();
  }, [refreshCart]);

  const addItem = useCallback(async (productId, quantity = 1) => {
    await cartApi.addCartItem(productId, quantity);
    await refreshCart();
  }, [refreshCart]);

  const updateItem = useCallback(async (productId, quantity) => {
    await cartApi.updateCartItem(productId, quantity);
    await refreshCart();
  }, [refreshCart]);

  const removeItem = useCallback(async (productId) => {
    await cartApi.removeCartItem(productId);
    await refreshCart();
  }, [refreshCart]);

  const clear = useCallback(async () => {
    await cartApi.clearCart();
    await refreshCart();
  }, [refreshCart]);

  const itemCount = items.reduce((sum, i) => sum + i.quantity, 0);

  const value = { items, subtotal, itemCount, loading, addItem, updateItem, removeItem, clear, refreshCart };

  return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error("useCart must be used within a CartProvider");
  return ctx;
}
```

```
import { Navigate, useLocation } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";

export default function ProtectedRoute({ children }) {
  const { isAuthenticated } = useAuth();
  const location = useLocation();

  if (!isAuthenticated) {
    return <Navigate to="/Login" state={{ from: location }} replace />;
  }
  return children;
}
```

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";

export default function AdminRoute({ children }) {
  const { isAuthenticated, isAdmin } = useAuth();

  if (!isAuthenticated || !isAdmin) {
    return <Navigate to="/admin/login" replace />;
  }
  return children;
}
```

```
export default function Spinner({ size = 24, className = "" }) {
  return (
    <div
      role="status"
      aria-label="Loading"
      className={`inline-block animate-spin rounded-full border-2 border-cargo/20 border-t-cargo ${className}`}
      style={{ width: size, height: size }}
    />
  );
}
```

```

export default function StarRating({ rating = 0, reviewCount, size = "text-sm", interactive = false, onChange }) {
  const stars = [1, 2, 3, 4, 5];

  return (
    <div className={`flex items-center gap-1 ${size}`}>
      <div className="flex" role={interactive ? "radiogroup" : "img"} aria-label={` ${rating} out of 5 stars`}>
        {stars.map((star) => (
          <button
            key={star}
            type="button"
            disabled={!interactive}
            onClick={() => interactive && onChange?.(star)}
            className={` ${interactive ? "cursor-pointer hover:scale-110 transition-transform" : "cursor-default"} ${
              star <= Math.round(rating) ? "text-crate" : "text-line"
            }`}
            aria-label={` ${star} star${star > 1 ? "s" : ""}`}
          >
            *
          </button>
        ))}
      </div>
      {reviewCount !== undefined && <span className="text-slate text-xs">({reviewCount})</span>}
    </div>
  );
}

```

```

export default function Pagination({ page, totalPages, onChange }) {
  if (totalPages <= 1) return null;

  const pages = Array.from({ length: totalPages }, (_, i) => i + 1);

  return (
    <nav className="flex items-center justify-center gap-1.5 py-8" aria-label="Pagination">
      <button
        className="btn-ghost !px-3 !py-1.5"
        onClick={() => onChange(Math.max(1, page - 1))}
        disabled={page === 1}
      >
        Prev
      </button>
      {pages.map((p) => (
        <button
          key={p}
          onClick={() => onChange(p)}
          aria-current={p === page ? "page" : undefined}
          className={`h-9 w-9 rounded-full text-sm font-mono font-semibold transition-colors ${
            p === page ? "bg-cargo text-white" : "text-ink hover:bg-mist"
          }`}
        >
          {p}
        </button>
      ))}
      <button
        className="btn-ghost !px-3 !py-1.5"
        onClick={() => onChange(Math.min(totalPages, page + 1))}
        disabled={page === totalPages}
      >
        Next
      </button>
    </nav>
  );
}

```

```
export default function EmptyState({ icon = "🤖", title, message, action }) {  
  return (  
    <div className="flex flex-col items-center justify-center rounded-card border border-dashed border-line bg-paper/50 py-16 px-6 text-center">  
      <div className="mb-3 text-4xl" aria-hidden="true">{icon}</div>  
      <h3 className="text-lg font-display font-semibold text-ink">{title}</h3>  
      {message && <p className="mt-1.5 max-w-sm text-sm text-slate">{message}</p>}  
      {action && <div className="mt-5">{action}</div>}  
    </div>  
  );  
}
```

```
const STYLES = {
  PLACED: "badge-warn",
  PROCESSING: "badge-warn",
  SHIPPED: "badge-ok",
  DELIVERED: "badge-ok",
  CANCELLED: "badge-danger",
  ACTIVE: "badge-ok",
  BLOCKED: "badge-danger",
};

export default function StatusBadge({ status }) {
  return <span className={STYLES[status] || "badge bg-slate/10 text-slate"}>{status}</span>;
}
```

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";
import { useCart } from "../../context/CartContext";

export default function Navbar() {
  const [query, setQuery] = useState("");
  const navigate = useNavigate();
  const { user, isAuthenticated, logout } = useAuth();
  const { itemCount } = useCart();

  const handleSearch = (e) => {
    e.preventDefault();
    navigate(query.trim() ? `/products?search=${encodeURIComponent(query.trim())}` : "/products");
  };

  return (
    <header className="sticky top-0 z-40 border-b border-line bg-paper/95 backdrop-blur">
      <div className="mx-auto flex max-w-7xl items-center gap-6 px-4 py-3 sm:px-6 lg:px-8">
        <Link to="/" className="flex items-center gap-2 shrink-0">
          <svg width="28" height="28" viewBox="0 0 28 28" fill="none" aria-hidden="true">
            <rect x="3" y="14" width="9" height="9" rx="1.5" fill="#4338CA" />
            <rect x="14" y="9" width="9" height="9" rx="1.5" fill="#FF6B4A" />
            <rect x="8.5" y="4" width="9" height="9" rx="1.5" fill="#4338CA" opacity="0.35" />
          </svg>
          <span className="font-display text-lg font-bold text-ink">CloudCart</span>
        </Link>

        <form onSubmit={handleSearch} className="hidden flex-1 max-w-md md:block">
          <input
            type="search"
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Search products..."
            className="input"
            aria-label="Search products"
          />
        </form>

        <nav className="hidden items-center gap-5 text-sm font-medium text-ink lg:flex">
          <Link to="/products" className="hover:text-cargo">Shop</Link>
          <Link to="/wishlist" className="hover:text-cargo">Wishlist</Link>
          {isAuthenticated && <Link to="/orders" className="hover:text-cargo">Orders</Link>}
        </nav>

        <div className="ml-auto flex items-center gap-3">
          <Link to="/cart" className="relative flex h-10 w-10 items-center justify-center rounded-full hover:bg-mist" aria-label="View cart">
            <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
              <path d="M3 3h2l2.4 12.2a2 2 0 002 1.8h8.2a2 2 0 002-1.6L21 8H6" />
              <circle cx="9" cy="20" r="1.4" fill="currentColor" />
              <circle cx="18" cy="20" r="1.4" fill="currentColor" />
            </svg>
            {itemCount > 0 && (
              <span key={itemCount} className="absolute -top-1 -right-1 flex h-5 w-5 animate-stackdrop items-center justify-center rounded-full bg-crate text-[11px] font-mono font-bold text-white">
                {itemCount}
              </span>
            )}
          </Link>

          {isAuthenticated ? (
            <div className="hidden items-center gap-3 sm:flex">
              <Link to="/profile" className="text-sm font-medium text-ink hover:text-cargo">{user.name.split(" ")[0]}</Link>
              <button onClick={logout} className="btn-ghost !px-3 !py-1.5 text-xs">Log out</button>
            </div>
          ) : (
            <Link to="/login" className="btn-primary !px-4 !py-2 text-xs">Log in</Link>
          )}
        </div>
      </div>

      <form onSubmit={handleSearch} className="px-4 pb-3 md:hidden">
        <input
          type="search"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Search products..."
          className="input"
          aria-label="Search products"
        />
      </form>
    </header>
  );
}

```

```

export default function Footer() {
  return (
    <footer className="mt-16 border-t border-line bg-paper">
      <div className="mx-auto max-w-7xl px-4 py-10 text-sm text-slate sm:px-6 lg:px-8">
        <div className="grid grid-cols-2 gap-8 sm:grid-cols-4">
          <div>
            <p className="eyebrow mb-3">Shop</p>
            <ul className="space-y-2">
              <li>All products</li>
              <li>Deals</li>
              <li>New arrivals</li>
            </ul>
          </div>
          <div>
            <p className="eyebrow mb-3">Account</p>
            <ul className="space-y-2">
              <li>Orders</li>
              <li>Wishlist</li>
              <li>Profile</li>
            </ul>
          </div>
          <div>
            <p className="eyebrow mb-3">Support</p>
            <ul className="space-y-2">
              <li>Shipping info</li>
              <li>Returns</li>
              <li>Contact us</li>
            </ul>
          </div>
          <div>
            <p className="eyebrow mb-3">Company</p>
            <ul className="space-y-2">
              <li>About CloudCart</li>
              <li>Careers</li>
            </ul>
          </div>
        </div>
        <p className="mt-10 border-t border-line pt-6 text-xs text-slate">
          © {new Date().getFullYear()} CloudCart. Built on AWS.
        </p>
      </div>
    </footer>
  );
}

```

```
import { Outlet } from "react-router-dom";
import Navbar from "../Navbar";
import Footer from "../Footer";

export default function StorefrontLayout() {
  return (
    <div className="flex min-h-screen flex-col">
      <Navbar />
      <main className="flex-1">
        <Outlet />
      </main>
      <Footer />
    </div>
  );
}
```

```

import { NavLink } from "react-router-dom";

const LINKS = [
  { to: "/admin/dashboard", label: "Dashboard", icon: "■" },
  { to: "/admin/products", label: "Products", icon: "■" },
  { to: "/admin/categories", label: "Categories", icon: "■" },
  { to: "/admin/inventory", label: "Inventory", icon: "■" },
  { to: "/admin/orders", label: "Orders", icon: "■" },
  { to: "/admin/customers", label: "Customers", icon: "◆" },
  { to: "/admin/reports", label: "Reports", icon: "■" },
];

export default function AdminSidebar() {
  return (
    <aside className="hidden w-60 shrink-0 flex-col border-r border-white/10 bg-ink text-white md:flex">
      <div className="flex items-center gap-2 px-5 py-5">
        <svg width="22" height="22" viewBox="0 0 28 28" fill="none" aria-hidden="true">
          <rect x="3" y="14" width="9" height="9" rx="1.5" fill="#818CF8" />
          <rect x="14" y="9" width="9" height="9" rx="1.5" fill="#FF6B4A" />
          <rect x="8.5" y="4" width="9" height="9" rx="1.5" fill="#818CF8" opacity="0.5" />
        </svg>
        <span className="font-display text-base font-bold">CloudCart <span className="text-white/40 font-mono text-xs align-middle">/admin</span></span>
      </div>
      <nav className="flex-1 space-y-0.5 px-3 py-2">
        {LINKS.map((link) => (
          <NavLink
            key={link.to}
            to={link.to}
            className={({ isActive }) =>
              `flex items-center gap-3 rounded-lg px-3 py-2.5 text-sm font-medium transition-colors ${
                isActive ? "bg-white/10 text-white" : "text-white/60 hover:bg-white/5 hover:text-white"
              }`}
          >
            <span className="font-mono text-base leading-none" aria-hidden="true">{link.icon}</span>
            {link.label}
          </NavLink>
        ))}
      </nav>
      <div className="border-t border-white/10 px-5 py-4 font-mono text-[11px] text-white/40">
        CloudCart Admin v1.0
      </div>
    </aside>
  );
}

```

```
import { useNavigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";

export default function AdminTopbar({ title }) {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = async () => {
    await logout();
    navigate("/admin/login");
  };

  return (
    <header className="flex items-center justify-between border-b border-line bg-paper px-6 py-4">
      <h1 className="font-display text-xl font-semibold text-ink">{title}</h1>
      <div className="flex items-center gap-4">
        <span className="text-sm text-slate">{user?.name}</span>
        <button onClick={handleLogout} className="btn-ghost !px-3 !py-1.5 text-xs">Log out</button>
      </div>
    </header>
  );
}
```

```
import { Outlet } from "react-router-dom";
import AdminSidebar from "../AdminSidebar";

export default function AdminLayout() {
  return (
    <div className="flex min-h-screen bg-mist">
      <AdminSidebar />
      <div className="flex-1 min-w-0">
        <Outlet />
      </div>
    </div>
  );
}
```

```

// CloudCart's signature: until real product photography is uploaded to S3,
// each product renders as a stacked-crate glyph on a soft gradient tinted by
// its category, keeping the catalog visually coherent rather than using
// generic gray placeholder boxes.
const PALETTE = [
  ["#EEF0FD", "#4338CA"],
  ["#FFE9E1", "#E5502F"],
  ["#E7F8EE", "#16A34A"],
  ["#FEF3E2", "#D97706"],
];

function hashToIndex(str, mod) {
  let hash = 0;
  for (let i = 0; i < str.length; i++) hash = (hash * 31 + str.charCodeAt(i)) % 997;
  return hash % mod;
}

export default function ProductImage({ seed = "product", className = "" }) {
  const [bg, fg] = PALETTE[hashToIndex(seed, PALETTE.length)];

  return (
    <div
      className={`flex items-center justify-center rounded-card ${className}`}
      style={{ background: `linear-gradient(160deg, ${bg} 0%, #FFFFFF 100%)` }}
      aria-hidden="true"
    >
      <svg width="46%" viewBox="0 0 64 64" fill="none">
        <rect x="10" y="28" width="18" height="18" rx="2" stroke={fg} strokeWidth="2.5" />
        <rect x="30" y="20" width="18" height="18" rx="2" stroke={fg} strokeWidth="2.5" />
        <rect x="20" y="12" width="18" height="18" rx="2" stroke={fg} strokeWidth="2.5" opacity="0.5" />
      </svg>
    </div>
  );
}

```

```

import { Link } from "react-router-dom";
import ProductImage from "../ProductImage";
import StarRating from "../common/StarRating";

export default function ProductCard({ product, onAddToCart, onToggleWishlist, isWishlisted }) {
  const lowStock = product.stock > 0 && product.stock < 10;
  const outOfStock = product.stock === 0;

  return (
    <div className="card group relative flex flex-col overflow-hidden">
      <button
        onClick={() => onToggleWishlist?.(product.id)}
        aria-label={isWishlisted ? "Remove from wishlist" : "Add to wishlist"}
        aria-pressed={isWishlisted}
        className="absolute right-3 top-3 z-10 flex h-8 w-8 items-center justify-center rounded-full bg-white/90 text-base shadow-sm transition-transform hover:scale-110"
      >
        {isWishlisted ? "♥" : "♡"}
      </button>

      <Link to={` /products/${product.id}`} className="block aspect-square p-4">
        <ProductImage seed={product.image} className="h-full w-full" />
      </Link>

      <div className="flex flex-1 flex-col gap-2 border-t border-line p-4">
        <Link to={` /products/${product.id}`} className="font-display text-sm font-semibold text-ink hover:text-cargo line-clamp-2">
          {product.name}
        </Link>
        <StarRating rating={product.rating} reviewCount={product.reviewCount} />
        <div className="mt-auto flex items-center justify-between pt-2">
          <span className="price text-lg">${product.price.toFixed(2)}</span>
          {outOfStock ? (
            <span className="badge-danger">Out of stock</span>
          ) : lowStock ? (
            <span className="badge-warn">Only {product.stock} left</span>
          ) : (
            <span className="badge-ok">In stock</span>
          )}
        </div>
        <button
          onClick={() => onAddToCart?.(product.id)}
          disabled={outOfStock}
          className="btn-primary mt-2 w-full !py-2 text-xs"
        >
          {outOfStock ? "Unavailable" : "Add to cart"}
        </button>
      </div>
    </div>
  );
}

```

```
import ProductCard from "../ProductCard";
import EmptyState from "../common/EmptyState";

export default function ProductGrid({ products, onAddToCart, onToggleWishlist, wishlistIds = [] }) {
  if (!products || products.length === 0) {
    return <EmptyState icon="🔍" title="No products found" message="Try a different search term or clear your filters." />;
  }

  return (
    <div className="grid grid-cols-2 gap-4 sm:grid-cols-3 lg:grid-cols-4">
      {products.map((product) => (
        <ProductCard
          key={product.id}
          product={product}
          onAddToCart={onAddToCart}
          onToggleWishlist={onToggleWishlist}
          isWishlisted={wishlistIds.includes(product.id)}
        />
      ))}
    </div>
  );
}
```

```
export default function CategoryFilter({ categories, selectedId, onSelect }) {
  return (
    <div className="flex flex-wrap gap-2 lg:flex-col lg:flex-nowrap">
      <button
        onClick={() => onSelect(null)}
        className={`rounded-full px-4 py-2 text-sm font-medium transition-colors lg:rounded-lg lg:text-left ${
          !selectedId ? "bg-cargo text-white" : "bg-paper text-ink border border-line hover:bg-mist"
        }`}
      >
        All products
      </button>
      {categories.map((cat) => (
        <button
          key={cat.id}
          onClick={() => onSelect(cat.id)}
          className={`rounded-full px-4 py-2 text-sm font-medium transition-colors lg:rounded-lg lg:text-left ${
            selectedId === cat.id ? "bg-cargo text-white" : "bg-paper text-ink border border-line hover:bg-mist"
          }`}
        >
          {cat.name}
        </button>
      ))}
    </div>
  );
}
```

```
import StarRating from "../common/StarRating";
import EmptyState from "../common/EmptyState";

export default function ReviewList({ reviews }) {
  if (!reviews || reviews.length === 0) {
    return <EmptyState icon="🗉" title="No reviews yet" message="Be the first to share what you think of this product." />;
  }

  return (
    <ul className="space-y-4">
      {reviews.map((review) => (
        <li key={review.id} className="card p-4">
          <div className="flex items-center justify-between">
            <span className="font-medium text-ink text-sm">{review.userName}</span>
            <span className="text-xs text-slate">{review.date}</span>
          </div>
          <div className="mt-1">
            <StarRating rating={review.rating} />
          </div>
          <p className="mt-2 text-sm text-slate">{review.comment}</p>
        </li>
      ))}
    </ul>
  );
}
```

```

import { useState } from "react";
import StarRating from "../common/StarRating";

export default function ReviewForm({ onSubmit }) {
  const [rating, setRating] = useState(0);
  const [comment, setComment] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (rating === 0) {
      setError("Please select a star rating.");
      return;
    }
    setError("");
    setSubmitting(true);
    try {
      await onSubmit({ rating, comment });
      setRating(0);
      setComment("");
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="card space-y-3 p-4">
      <p className="label">Your rating</p>
      <StarRating rating={rating} interactive onChange={setRating} size="text-xl" />
      <textarea
        value={comment}
        onChange={(e) => setComment(e.target.value)}
        placeholder="Share details about your experience with this product..."
        rows={3}
        className="input"
      />
      {error && <p className="text-sm text-danger">{error}</p>}
      <button type="submit" disabled={!submitting} className="btn-primary !py-2 text-xs">
        {submitting ? "Posting..." : "Post review"}
      </button>
    </form>
  );
}

```

```

import ProductImage from "../product/ProductImage";

export default function CartItemRow({ item, onUpdateQuantity, onRemove }) {
  return (
    <div className="flex items-center gap-4 border-b border-line py-4 last:border-0">
      <div className="h-20 w-20 shrink-0">
        <ProductImage seed={item.image} className="h-full w-full" />
      </div>
      <div className="min-w-0 flex-1">
        <p className="truncate font-medium text-ink text-sm">{item.name}</p>
        <p className="price mt-1 text-sm">${item.price.toFixed(2)}</p>
      </div>
      <div className="flex items-center gap-2">
        <button
          onClick={() => onUpdateQuantity(item.productId, Math.max(1, item.quantity - 1))}
          className="h-8 w-8 rounded-full border border-line text-sm hover:bg-mist"
          aria-label="Decrease quantity"
        >
          -
        </button>
        <span className="w-6 text-center font-mono text-sm">{item.quantity}</span>
        <button
          onClick={() => onUpdateQuantity(item.productId, Math.min(item.stock, item.quantity + 1))}
          className="h-8 w-8 rounded-full border border-line text-sm hover:bg-mist"
          aria-label="Increase quantity"
        >
          +
        </button>
      </div>
      <p className="w-20 text-right price text-sm">${(item.price * item.quantity).toFixed(2)}</p>
      <button
        onClick={() => onRemove(item.productId)}
        className="text-slate hover:text-danger"
        aria-label={`Remove ${item.name} from cart`}
      >
        x
      </button>
    </div>
  );
}

```

```

import { Link } from "react-router-dom";

export default function CartSummary({ subtotal, shipping = 0, checkoutTo = "/checkout", checkoutLabel = "Proceed to checkout" }) {
  const total = subtotal + shipping;

  return (
    <div className="card p-5">
      <h3 className="font-display text-base font-semibold text-ink">Order summary</h3>
      <div className="mt-4 space-y-2 text-sm">
        <div className="flex justify-between text-slate">
          <span>Subtotal</span>
          <span className="font-mono">${subtotal.toFixed(2)}</span>
        </div>
        <div className="flex justify-between text-slate">
          <span>Shipping</span>
          <span className="font-mono">{shipping === 0 ? "Free" : ` ${shipping.toFixed(2)}`}</span>
        </div>
        <div className="flex justify-between border-t border-line pt-2 font-semibold text-ink">
          <span>Total</span>
          <span className="font-mono">${total.toFixed(2)}</span>
        </div>
      </div>
      <Link to={checkoutTo} className="btn-accent mt-5 w-full">
        {checkoutLabel}
      </Link>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import * as productApi from "../../api/productApi";
import * as categoryApi from "../../api/categoryApi";
import * as cartApi from "../../api/cartApi";
import * as wishlistApi from "../../api/wishlistApi";
import { useCart } from "../../context/CartContext";
import ProductGrid from "../../components/product/ProductGrid";
import Spinner from "../../components/common/Spinner";

export default function Home() {
  const [categories, setCategories] = useState([]);
  const [featured, setFeatured] = useState([]);
  const [wishlistIds, setWishlistIds] = useState([]);
  const [loading, setLoading] = useState(true);
  const { refreshCart } = useCart();

  useEffect(() => {
    (async () => {
      const [cats, products, wishlist] = await Promise.all([
        categoryApi.getCategories(),
        productApi.getProducts({ page: 1, size: 8 }),
        wishlistApi.getWishlist(),
      ]);
      setCategories(cats);
      setFeatured(products.content);
      setWishlistIds(wishlist.map((p) => p.id));
      setLoading(false);
    })();
  }, []);

  const handleAddToCart = async (productId) => {
    await cartApi.addCartItem(productId, 1);
    await refreshCart();
  };

  const handleToggleWishlist = async (productId) => {
    if (wishlistIds.includes(productId)) {
      await wishlistApi.removeFromWishlist(productId);
      setWishlistIds((ids) => ids.filter((id) => id !== productId));
    } else {
      await wishlistApi.addToWishlist(productId);
      setWishlistIds((ids) => [...ids, productId]);
    }
  };

  return (
    <div>
      { /* Hero: the signature moment – a shipment "arriving" as stacked crates */ }
      <section className="border-b border-line bg-gradient-to-br from-cargo-light to-mist">
        <div className="mx-auto grid max-w-7xl items-center gap-10 px-4 py-16 sm:px-6 lg:grid-cols-2 lg:px-8 lg:py-24">
          <div>
            <p className="eyebrow">Cloud-native retail</p>
            <h1 className="mt-3 text-4xl font-bold leading-tight text-ink sm:text-5xl">
              Shop smarter.<br />Shipped from the cloud.
            </h1>
            <p className="mt-4 max-w-md text-slate">
              Thousands of products, real-time inventory, and orders that ship the moment you check out –
              running on infrastructure built to never go down.
            </p>
            <div className="mt-7 flex gap-3">
              <Link to="/products" className="btn-accent">Browse products</Link>
              <Link to="/register" className="btn-ghost bg-white">Create an account</Link>
            </div>
          </div>
          <div className="relative hidden h-64 items-center justify-center lg:flex">
            <svg width="280" height="220" viewBox="0 0 280 220" fill="none" aria-hidden="true">
              <rect x="20" y="120" width="90" height="80" rx="8" fill="#4338CA" opacity="0.9" />
              <rect x="120" y="90" width="90" height="80" rx="8" fill="#FF6B4A" />
              <rect x="70" y="40" width="90" height="80" rx="8" fill="#4338CA" opacity="0.35" />
              <rect x="170" y="140" width="70" height="60" rx="8" fill="#4338CA" opacity="0.6" />
            </svg>
          </div>
        </div>
      </section>

      <section className="mx-auto max-w-7xl px-4 py-10 sm:px-6 lg:px-8">
        <h2 className="text-xl font-semibold text-ink">Shop by category</h2>
        <div className="mt-4 flex flex-wrap gap-3">
          {categories.map((cat) => (
            <Link key={cat.id} to={`/products?categoryId=${cat.id}`} className="card px-5 py-3 text-sm font-medium text-ink hover:border-cargo hover:text-cargo">
              {cat.name}
            </Link>
          ))}
        </div>
      </section>

      <section className="mx-auto max-w-7xl px-4 pb-16 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between">
          <h2 className="text-xl font-semibold text-ink">Trending now</h2>
          <Link to="/products" className="text-sm font-medium text-cargo hover:underline">View all </Link>
        </div>
        <div className="mt-4">
          {loading ? (
            <div className="flex justify-center py-16"><Spinner size={32} /></div>
          ) : (
            <ProductGrid products={featured} onAddToCart={handleAddToCart} onToggleWishlist={handleToggleWishlist} wishlistIds={wishlistIds} />
          )}
        </div>
      </section>
    </div>
  );
}

```

```

import { useState } from "react";
import { Link, useNavigate, useLocation } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";

export default function Login() {
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setSubmitting(true);
    try {
      await login(form);
      navigate(location.state?.from?.pathname || "/");
    } catch (err) {
      setError(err.message);
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="mx-auto flex min-h-[70vh] max-w-md flex-col justify-center px-4 py-16">
      <h1 className="text-2xl font-bold text-ink">Welcome back</h1>
      <p className="mt-1 text-sm text-slate">Log in to continue to CloudCart.</p>

      <form onSubmit={handleSubmit} className="mt-8 space-y-4">
        <div>
          <label className="label" htmlFor="email">Email address</label>
          <input
            id="email"
            type="email"
            required
            className="input"
            value={form.email}
            onChange={(e) => setForm({ ...form, email: e.target.value })}
          />
        </div>
        <div>
          <label className="label" htmlFor="password">Password</label>
          <input
            id="password"
            type="password"
            required
            className="input"
            value={form.password}
            onChange={(e) => setForm({ ...form, password: e.target.value })}
          />
        </div>
        {error && <p className="text-sm text-danger">{error}</p>}
        <button type="submit" disabled={submitting} className="btn-primary w-full">
          {submitting ? "Logging in..." : "Log in"}
        </button>
      </form>

      <div className="mt-4 flex justify-between text-sm">
        <Link to="/forgot-password" className="text-cargo hover:underline">Forgot password?</Link>
        <Link to="/register" className="text-cargo hover:underline">Create an account</Link>
      </div>

      <p className="mt-6 rounded-lg bg-cargo-light px-4 py-3 text-xs text-cargo">
        Demo login: <span className="font-mono">asha@example.com</span> / <span className="font-mono">Password@123</span>
      </p>
    </div>
  );
}

```

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";

export default function Register() {
  const [form, setForm] = useState({ name: "", email: "", password: "", confirmPassword: "" });
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const { register } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    if (form.password !== form.confirmPassword) {
      setError("Passwords do not match.");
      return;
    }
    if (form.password.length < 8) {
      setError("Password must be at least 8 characters.");
      return;
    }
    setSubmitting(true);
    try {
      await register(form);
      setSuccess("Account created! Redirecting to log in...");
      setTimeout(() => navigate("/login"), 1200);
    } catch (err) {
      setError(err.message);
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="mx-auto flex min-h-[70vh] max-w-md flex-col justify-center px-4 py-16">
      <h1 className="text-2xl font-bold text-ink">Create your account</h1>
      <p className="mt-1 text-sm text-slate">Join CloudCart to track orders and save your wishlist.</p>

      <form onSubmit={handleSubmit} className="mt-8 space-y-4">
        <div>
          <label className="label" htmlFor="name">Full name</label>
          <input id="name" required className="input" value={form.name} onChange={(e) => setForm({ ...form, name: e.target.value })} />
        </div>
        <div>
          <label className="label" htmlFor="email">Email address</label>
          <input id="email" type="email" required className="input" value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} />
        </div>
        <div>
          <label className="label" htmlFor="password">Password</label>
          <input id="password" type="password" required className="input" value={form.password} onChange={(e) => setForm({ ...form, password: e.target.value })} />
        </div>
        <div>
          <label className="label" htmlFor="confirmPassword">Confirm password</label>
          <input id="confirmPassword" type="password" required className="input" value={form.confirmPassword} onChange={(e) => setForm({ ...form, confirmPassword: e.target.value })} />
        </div>
        {error && <p className="text-sm text-danger">{error}</p>}
        {success && <p className="text-sm text-ok">{success}</p>}
        <button type="submit" disabled={submitting} className="btn-primary w-full">
          {submitting ? "Creating account..." : "Create account"}
        </button>
      </form>

      <p className="mt-4 text-center text-sm text-slate">
        Already have an account? <Link to="/login" className="text-cargo hover:underline">Log in</Link>
      </p>
    </div>
  );
}

```

```

import { useState } from "react";
import { Link } from "react-router-dom";
import * as authApi from "../../api/authApi";

export default function ForgotPassword() {
  const [email, setEmail] = useState("");
  const [message, setMessage] = useState("");
  const [submitting, setSubmitting] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setSubmitting(true);
    try {
      const res = await authApi.forgotPassword(email);
      setMessage(res.message);
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="mx-auto flex min-h-[70vh] max-w-md flex-col justify-center px-4 py-16">
      <h1 className="text-2xl font-bold text-ink">Reset your password</h1>
      <p className="mt-1 text-sm text-slate">Enter your email and we'll send you a reset link.</p>

      <form onSubmit={handleSubmit} className="mt-8 space-y-4">
        <div>
          <label className="label" htmlFor="email">Email address</label>
          <input id="email" type="email" required className="input" value={email} onChange={(e) => setEmail(e.target.value)} />
        </div>
        {message && <p className="text-sm text-ok">{message}</p>}
        <button type="submit" disabled={submitting} className="btn-primary w-full">
          {submitting ? "Sending..." : "Send reset link"}
        </button>
      </form>

      <p className="mt-4 text-center text-sm">
        <Link to="/login" className="text-cargo hover:underline">Back to log in</Link>
      </p>
    </div>
  );
}

```

```

import { useEffect, useState, useCallback } from "react";
import { useSearchParams } from "react-router-dom";
import * as productApi from "../../api/productApi";
import * as categoryApi from "../../api/categoryApi";
import * as wishlistApi from "../../api/wishlistApi";
import * as cartApi from "../../api/cartApi";
import { useCart } from "../../context/CartContext";
import ProductGrid from "../../components/product/ProductGrid";
import CategoryFilter from "../../components/product/CategoryFilter";
import Pagination from "../../components/common/Pagination";
import Spinner from "../../components/common/Spinner";

export default function Products() {
  const [searchParams, setSearchParams] = useSearchParams();
  const search = searchParams.get("search") || "";
  const categoryId = searchParams.get("categoryId") ? Number(searchParams.get("categoryId")) : null;
  const page = Number(searchParams.get("page") || 1);

  const [categories, setCategories] = useState([]);
  const [result, setResult] = useState({ content: [], totalPages: 1 });
  const [wishlistIds, setWishlistIds] = useState([]);
  const [loading, setLoading] = useState(true);
  const { refreshCart } = useCart();

  const loadProducts = useCallback(async () => {
    setLoading(true);
    const data = await productApi.getProducts({ search, categoryId, page, size: 12 });
    setResult(data);
    setLoading(false);
  }, [search, categoryId, page]);

  useEffect(() => {
    categoryApi.getCategories().then(setCategories);
    wishlistApi.getWishlist().then((list) => setWishlistIds(list.map((p) => p.id)));
  }, []);

  useEffect(() => {
    loadProducts();
  }, [loadProducts]);

  const updateParams = (updates) => {
    const next = new URLSearchParams(searchParams);
    Object.entries(updates).forEach(([key, value]) => {
      if (value === null || value === "") next.delete(key);
      else next.set(key, value);
    });
    setSearchParams(next);
  };

  const handleAddToCart = async (productId) => {
    await cartApi.addCartItem(productId, 1);
    await refreshCart();
  };

  const handleToggleWishlist = async (productId) => {
    if (wishlistIds.includes(productId)) {
      await wishlistApi.removeFromWishlist(productId);
      setWishlistIds((ids) => ids.filter((id) => id !== productId));
    } else {
      await wishlistApi.addToWishlist(productId);
      setWishlistIds((ids) => [...ids, productId]);
    }
  };

  return (
    <div className="mx-auto max-w-7xl px-4 py-8 sm:px-6 lg:px-8">
      <div className="mb-6 flex items-center justify-between">
        <h1 className="text-2xl font-bold text-ink">{search ? `Results for "${search}"` : "All products"}</h1>
        <span className="text-sm text-slate font-mono">{result.totalElements ?? result.content.length} items</span>
      </div>

      <div className="grid grid-cols-1 gap-8 lg:grid-cols-[220px_1fr]">
        <CategoryFilter
          categories={categories}
          selectedId={categoryId}
          onSelect={(id) => updateParams({ categoryId: id, page: null })}
        />

        <div>
          {loading ? (
            <div className="flex justify-center py-20"><Spinner size={32} /></div>
          ) : (
            <ProductGrid
              products={result.content}
              onAddToCart={handleAddToCart}
              onToggleWishlist={handleToggleWishlist}
              wishlistIds={wishlistIds}
            />
            <Pagination page={page} totalPages={result.totalPages} onChange={(p) => updateParams({ page: p })} />
          )}
        </div>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import { useParams, Link } from "react-router-dom";
import * as productApi from "../../api/productApi";
import * as reviewApi from "../../api/reviewApi";
import * as wishlistApi from "../../api/wishlistApi";
import * as cartApi from "../../api/cartApi";
import { useCart } from "../../context/CartContext";
import { useAuth } from "../../context/AuthContext";
import ProductImage from "../../components/product/ProductImage";
import StarRating from "../../components/common/StarRating";
import ReviewList from "../../components/product/ReviewList";
import ReviewForm from "../../components/product/ReviewForm";
import Spinner from "../../components/common/Spinner";

export default function ProductDetails() {
  const { id } = useParams();
  const [product, setProduct] = useState(null);
  const [reviews, setReviews] = useState([]);
  const [quantity, setQuantity] = useState(1);
  const [isWishlisted, setIsWishlisted] = useState(false);
  const [addedMessage, setAddedMessage] = useState("");
  const [loading, setLoading] = useState(true);
  const { refreshCart } = useCart();
  const { user } = useAuth();

  useEffect(() => {
    setLoading(true);
    Promise.all([productApi.getProductById(id), reviewApi.getReviews(id), wishlistApi.getWishlist()]).then(
      ([p, r, wishlist]) => {
        setProduct(p);
        setReviews(r);
        setIsWishlisted(wishlist.some((w) => w.id === Number(id)));
        setLoading(false);
      }
    );
  }, [id]);

  const handleAddToCart = async () => {
    await cartApi.addCartItem(product.id, quantity);
    await refreshCart();
    setAddedMessage(`Added ${quantity} to your cart.`);
    setTimeout(() => setAddedMessage(""), 2500);
  };

  const handleToggleWishlist = async () => {
    if (isWishlisted) await wishlistApi.removeFromWishlist(product.id);
    else await wishlistApi.addToWishlist(product.id);
    setIsWishlisted(!isWishlisted);
  };

  const handleAddReview = async ({ rating, comment }) => {
    const review = await reviewApi.addReview(id, { rating, comment, userName: user?.name });
    setReviews((prev) => [...prev, review]);
  };

  if (loading) return <div className="flex justify-center py-24"><Spinner size={32} /></div>;
  if (!product) return <div className="py-24 text-center text-slate">Product not found.</div>;

  const outOfStock = product.stock === 0;

  return (
    <div className="mx-auto max-w-6xl px-4 py-8 sm:px-6 lg:px-8">
      <nav className="mb-6 text-sm text-slate">
        <Link to="/products" className="hover:text-cargo">Shop</Link> / <span className="text-ink">{product.name}</span>
      </nav>

      <div className="grid grid-cols-1 gap-10 lg:grid-cols-2">
        <div className="aspect-square">
          <ProductImage seed={product.image} className="h-full w-full" />
        </div>

        <div>
          <h1 className="text-2xl font-bold text-ink sm:text-3xl">{product.name}</h1>
          <div className="mt-2"><StarRating rating={product.rating} reviewCount={product.reviewCount} size="text-base" /></div>
          <p className="price mt-4 text-3xl">${product.price.toFixed(2)}</p>
          <p className="mt-4 text-slate">{product.description}</p>

          <div className="mt-4">
            {outOfStock ? (
              <span className="badge-danger">Out of stock</span>
            ) : product.stock < 10 ? (
              <span className="badge-warn">Only {product.stock} left in stock</span>
            ) : (
              <span className="badge-ok">In stock</span>
            )}
          </div>

          <div className="mt-6 flex items-center gap-4">
            <div className="flex items-center gap-2">
              <button onClick={() => setQuantity((q) => Math.max(1, q - 1))} className="h-9 w-9 rounded-full border border-line hover:bg-mist"></button>
              <span className="w-8 text-center font-mono">{quantity}</span>
              <button onClick={() => setQuantity((q) => Math.min(product.stock, q + 1))} className="h-9 w-9 rounded-full border border-line hover:bg-mist">+</button>
            </div>
            <button onClick={handleAddToCart} disabled={outOfStock} className="btn-primary flex-1 !py-3">
              {outOfStock ? "Unavailable" : "Add to cart"}
            </button>
            <button onClick={handleToggleWishlist} aria-pressed={isWishlisted} className="btn-ghost !px-4 !py-3 text-lg">
              {isWishlisted ? "♥" : "♡"}
            </button>
          </div>
          {addedMessage && <p className="mt-3 text-sm text-ok">{addedMessage}</p>}
        </div>
      </div>

      <div className="mt-14 grid grid-cols-1 gap-8 lg:grid-cols-[1fr_320px]">
        <div>

```

```
    <h2 className="text-xl font-semibold text-ink">Customer reviews</h2>
    <div className="mt-4"><ReviewList reviews={reviews} /></div>
  </div>
  <div>
    <h2 className="mb-4 text-lg font-semibold text-ink">Write a review</h2>
    <ReviewForm onSubmit={handleAddReview} />
  </div>
</div>
</div>
);
}
```

```

import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import * as wishlistApi from "../../api/wishlistApi";
import * as cartApi from "../../api/cartApi";
import { useCart } from "../../context/CartContext";
import ProductGrid from "../../components/product/ProductGrid";
import Spinner from "../../components/common/Spinner";
import EmptyState from "../../components/common/EmptyState";

export default function Wishlist() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const { refreshCart } = useCart();

  const load = async () => {
    setLoading(true);
    setProducts(await wishlistApi.getWishlist());
    setLoading(false);
  };

  useEffect(() => {
    load();
  }, []);

  const handleAddToCart = async (productId) => {
    await cartApi.addCartItem(productId, 1);
    await refreshCart();
  };

  const handleRemove = async (productId) => {
    await wishlistApi.removeFromWishlist(productId);
    setProducts((prev) => prev.filter((p) => p.id !== productId));
  };

  return (
    <div className="mx-auto max-w-7xl px-4 py-8 sm:px-6 lg:px-8">
      <h1 className="text-2xl font-bold text-ink">Your wishlist</h1>
      <p className="mt-1 text-sm text-slate">Saved items you're keeping an eye on.</p>

      <div className="mt-6">
        {loading ? (
          <div className="flex justify-center py-20"><Spinner size={32} /></div>
        ) : products.length === 0 ? (
          <EmptyState
            icon="♡"
            title="Your wishlist is empty"
            message="Tap the heart icon on any product to save it here."
            action={<Link to="/products" className="btn-primary">Browse products</Link>}
          />
        ) : (
          <ProductGrid products={products} onAddToCart={handleAddToCart} onToggleWishlist={handleRemove} wishlistIds={products.map((p) => p.id)} />
        )}
      </div>
    </div>
  );
}

```

```

import { Link } from "react-router-dom";
import { useCart } from "../../context/CartContext";
import CartItemRow from "../../components/cart/CartItemRow";
import CartSummary from "../../components/cart/CartSummary";
import EmptyState from "../../components/common/EmptyState";
import Spinner from "../../components/common/Spinner";

export default function Cart() {
  const { items, subtotal, loading, updateItem, removeItem } = useCart();

  return (
    <div className="mx-auto max-w-5xl px-4 py-8 sm:px-6 lg:px-8">
      <h1 className="text-2xl font-bold text-ink">Your cart</h1>

      {loading ? (
        <div className="flex justify-center py-20"><Spinner size={32} /></div>
      ) : items.length === 0 ? (
        <div className="mt-8">
          <EmptyState
            icon="🛒"
            title="Your cart is empty"
            message="Add some products to get started."
            action={<Link to="/products" className="btn-primary">Browse products</Link>}
          />
        </div>
      ) : (
        <div className="mt-8 grid grid-cols-1 gap-8 lg:grid-cols-[1fr_320px]">
          <div className="card divide-y divide-line p-4">
            {items.map((item) => (
              <CartItemRow key={item.productId} item={item} onUpdateQuantity={updateItem} onRemove={removeItem} />
            ))}
          </div>
          <CartSummary subtotal={subtotal} />
        </div>
      )}
    </div>
  );
}

```

```

import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { useCart } from "../../context/CartContext";
import * as orderApi from "../../api/orderApi";

export default function Checkout() {
  const { items, subtotal, clear } = useCart();
  const navigate = useNavigate();
  const [address, setAddress] = useState({ fullName: "", line1: "", city: "", state: "", zip: "", phone: "" });
  const [paymentMethod, setPaymentMethod] = useState("CARD");
  const [placing, setPlacing] = useState(false);
  const [error, setError] = useState("");

  const handlePlaceOrder = async (e) => {
    e.preventDefault();
    setError("");
    setPlacing(true);
    try {
      const order = await orderApi.placeOrder({
        items: items.map((i) => ({ productId: i.productId, name: i.name, quantity: i.quantity, price: i.price })),
        shippingAddress: address,
        paymentMethod,
      });
      await clear();
      navigate(`/orders/${order.id}`, { state: { justPlaced: true } });
    } catch (err) {
      setError(err?.response?.data?.message || "Could not place your order. Please try again.");
    } finally {
      setPlacing(false);
    }
  };

  if (items.length === 0) {
    return (
      <div className="mx-auto max-w-lg px-4 py-24 text-center">
        <p className="text-slate">Your cart is empty. Add items before checking out.</p>
      </div>
    );
  }

  return (
    <div className="mx-auto max-w-5xl px-4 py-8 sm:px-6 lg:px-8">
      <h1 className="text-2xl font-bold text-ink">Checkout</h1>

      <form onSubmit={handlePlaceOrder} className="mt-8 grid grid-cols-1 gap-8 lg:grid-cols-[1fr_320px]">
        <div className="space-y-6">
          <div className="card p-5">
            <h2 className="font-display font-semibold text-ink">Shipping address</h2>
            <div className="mt-4 grid grid-cols-1 gap-4 sm:grid-cols-2">
              <div className="sm:col-span-2">
                <label className="label">Full name</label>
                <input required className="input" value={address.fullName} onChange={(e) => setAddress({ ...address, fullName: e.target.value })} />
              </div>
              <div className="sm:col-span-2">
                <label className="label">Address line</label>
                <input required className="input" value={address.line1} onChange={(e) => setAddress({ ...address, line1: e.target.value })} />
              </div>
              <div>
                <label className="label">City</label>
                <input required className="input" value={address.city} onChange={(e) => setAddress({ ...address, city: e.target.value })} />
              </div>
              <div>
                <label className="label">State</label>
                <input required className="input" value={address.state} onChange={(e) => setAddress({ ...address, state: e.target.value })} />
              </div>
              <div>
                <label className="label">ZIP code</label>
                <input required className="input" value={address.zip} onChange={(e) => setAddress({ ...address, zip: e.target.value })} />
              </div>
              <div>
                <label className="label">Phone</label>
                <input required className="input" value={address.phone} onChange={(e) => setAddress({ ...address, phone: e.target.value })} />
              </div>
            </div>
          </div>

          <div className="card p-5">
            <h2 className="font-display font-semibold text-ink">Payment method</h2>
            <div className="mt-4 space-y-2">
              {[
                { value: "CARD", label: "Credit / Debit card" },
                { value: "UPI", label: "UPI" },
                { value: "COD", label: "Cash on delivery" },
              ].map((option) => (
                <label key={option.value} className="flex items-center gap-3 rounded-lg border border-line px-4 py-3 text-sm has-[:checked]:border-cargo has-[:checked]:bg-cargo-light">
                  <input type="radio" name="payment" value={option.value} checked={paymentMethod === option.value} onChange={() => setPaymentMethod(option.value)} />
                  {option.label}
                </label>
              ))}
            </div>
          </div>

          {error && <p className="text-sm text-danger">{error}</p>}
        </div>

        <div className="card h-fit p-5">
          <h3 className="font-display font-semibold text-ink">Order total</h3>
          <div className="mt-4 flex justify-between text-sm text-slate">
            <span>Subtotal</span>
            <span className="font-mono">${subtotal.toFixed(2)}</span>
          </div>
        </div>
      </form>
    </div>
  );
}

```

```
    </div>
    <div className="mt-2 flex justify-between border-t border-line pt-2 font-semibold text-ink">
      <span>Total</span><span className="font-mono">${subtotal.toFixed(2)}</span>
    </div>
    <button type="submit" disabled={placing} className="btn-accent mt-5 w-full">
      {placing ? "Placing order..." : "Place order"}
    </button>
  </div>
</form>
</div>
);
}
```

```

import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import * as orderApi from "../../api/orderApi";
import StatusBadge from "../../components/common/StatusBadge";
import EmptyState from "../../components/common/EmptyState";
import Spinner from "../../components/common/Spinner";

export default function Orders() {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    orderApi.getOrders().then((data) => {
      setOrders(data);
      setLoading(false);
    });
  }, []);

  return (
    <div className="mx-auto max-w-4xl px-4 py-8 sm:px-6 lg:px-8">
      <h1 className="text-2xl font-bold text-ink">Your orders</h1>

      <div className="mt-6">
        {loading ? (
          <div className="flex justify-center py-20"><Spinner size={32} /></div>
        ) : orders.length === 0 ? (
          <EmptyState
            icon="🛒"
            title="No orders yet"
            message="Once you place an order, it'll show up here."
            action={<Link to="/products" className="btn-primary">Start shopping</Link>}
          />
        ) : (
          <div className="space-y-3">
            {orders.map((order) => (
              <Link key={order.id} to={`/orders/${order.id}`} className="card flex items-center justify-between p-4 hover:border-cargo">
                <div>
                  <p className="font-mono text-sm text-slate">Order #{order.id}</p>
                  <p className="text-xs text-slate">Placed {order.placedAt}</p>
                </div>
                <div className="text-right">
                  <p className="price">${order.total.toFixed(2)}</p>
                  <div className="mt-1"><StatusBadge status={order.status} /></div>
                </div>
              </Link>
            ))}
          </div>
        )}
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import { useParams, useLocation, Link } from "react-router-dom";
import * as orderApi from "../../api/orderApi";
import StatusBadge from "../../components/common/StatusBadge";
import Spinner from "../../components/common/Spinner";

export default function OrderDetails() {
  const { id } = useParams();
  const location = useLocation();
  const [order, setOrder] = useState(null);
  const [loading, setLoading] = useState(true);
  const [cancelling, setCancelling] = useState(false);
  const [note, setNote] = useState("");

  const load = async () => {
    setLoading(true);
    setOrder(await orderApi.getOrderById(id));
    setLoading(false);
  };

  useEffect(() => {
    load();
  }, [id]);

  const handleCancel = async () => {
    setCancelling(true);
    await orderApi.cancelOrder(id);
    await load();
    setCancelling(false);
  };

  const handleInvoice = async () => {
    const res = await orderApi.downloadInvoice(id);
    setNote(res.message || "Invoice downloaded.");
    setTimeout(() => setNote("", 3000);
  };

  if (loading) return <div className="flex justify-center py-24"><Spinner size={32} /></div>;
  if (!order) return <div className="py-24 text-center text-slate">Order not found.</div>;

  const canCancel = order.status === "PLACED" || order.status === "PROCESSING";

  return (
    <div className="mx-auto max-w-3xl px-4 py-8 sm:px-6 lg:px-8">
      <Link to="/orders" className="text-sm text-cargo hover:underline">← Back to orders</Link>

      {location.state?.justPlaced && (
        <div className="mt-4 rounded-lg bg-ok/10 px-4 py-3 text-sm text-ok">
          Your order has been placed! We'll email you shipping updates.
        </div>
      )}

      <div className="mt-4 flex items-center justify-between">
        <div>
          <h1 className="font-mono text-2xl font-bold text-ink">Order #{order.id}</h1>
          <p className="text-sm text-slate">Placed {order.placedAt}</p>
        </div>
        <StatusBadge status={order.status} />
      </div>

      <div className="card mt-6 divide-y divide-line p-5">
        {order.items.map((item) => (
          <div key={item.productId} className="flex justify-between py-3 text-sm first:pt-0 last:pb-0">
            <span className="text-ink">{item.name} × {item.quantity}</span>
            <span className="price">${(item.price * item.quantity).toFixed(2)}</span>
          </div>
        ))}
        <div className="flex justify-between pt-3 font-semibold text-ink">
          <span>Total</span>
          <span className="price">${order.total.toFixed(2)}</span>
        </div>
      </div>

      {order.shippingAddress && (
        <div className="card mt-4 p-5 text-sm text-slate">
          <p className="eyebrow mb-2">Shipping to</p>
          <p className="text-ink">{order.shippingAddress.fullName}</p>
          <p>{order.shippingAddress.line1}, {order.shippingAddress.city}, {order.shippingAddress.state} {order.shippingAddress.zip}</p>
        </div>
      )}

      <div className="mt-6 flex flex-wrap gap-3">
        <button onClick={handleInvoice} className="btn-ghost">Download invoice</button>
        {canCancel && (
          <button onClick={handleCancel} disabled={cancelling} className="btn-danger">
            {cancelling ? "Cancelling..." : "Cancel order"}
          </button>
        )}
      </div>
      {note && <p className="mt-3 text-sm text-slate">{note}</p>}
    </div>
  );
}

```

```

import { useState } from "react";
import { useAuth } from "../../context/AuthContext";

export default function Profile() {
  const { user } = useAuth();
  const [form, setForm] = useState({ name: user?.name || "", email: user?.email || "" });
  const [saved, setSaved] = useState(false);

  const handleSubmit = (e) => {
    e.preventDefault();
    // PUT /api/users/profile would be called here against the live backend.
    setSaved(true);
    setTimeout(() => setSaved(false), 2500);
  };

  return (
    <div className="mx-auto max-w-lg px-4 py-8 sm:px-6 lg:px-8">
      <h1 className="text-2xl font-bold text-ink">Your profile</h1>

      <form onSubmit={handleSubmit} className="card mt-6 space-y-4 p-5">
        <div>
          <label className="label">Full name</label>
          <input className="input" value={form.name} onChange={(e) => setForm({ ...form, name: e.target.value })} />
        </div>
        <div>
          <label className="label">Email address</label>
          <input type="email" className="input" value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} />
        </div>
        <div>
          <label className="label">Role</label>
          <input className="input bg-mist" value={user?.role} disabled />
        </div>
        {saved && <p className="text-sm text-ok">Profile updated.</p>}
        <button type="submit" className="btn-primary">Save changes</button>
      </form>
    </div>
  );
}

```

```
import { Link } from "react-router-dom";

export default function NotFound() {
  return (
    <div className="mx-auto flex min-h-[70vh] max-w-md flex-col items-center justify-center px-4 text-center">
      <p className="font-mono text-6xl font-bold text-cargo">404</p>
      <h1 className="mt-3 text-xl font-semibold text-ink">Page not found</h1>
      <p className="mt-1 text-sm text-slate">The page you're looking for doesn't exist or has moved.</p>
      <Link to="/" className="btn-primary mt-6">Back to home</Link>
    </div>
  );
}
```

```

import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";

export default function AdminLogin() {
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setSubmitting(true);
    try {
      const user = await login(form);
      if (user.role !== "ADMIN") {
        setError("This account does not have admin access.");
        return;
      }
      navigate("/admin/dashboard");
    } catch (err) {
      setError(err.message);
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="flex min-h-screen items-center justify-center bg-ink px-4">
      <div className="w-full max-w-sm">
        <div className="mb-6 flex items-center justify-center gap-2">
          <svg width="26" height="26" viewBox="0 0 28 28" fill="none" aria-hidden="true">
            <rect x="3" y="14" width="9" height="9" rx="1.5" fill="#818CF8" />
            <rect x="14" y="9" width="9" height="9" rx="1.5" fill="#FF6B4A" />
            <rect x="8.5" y="4" width="9" height="9" rx="1.5" fill="#818CF8" opacity="0.5" />
          </svg>
          <span className="font-display text-lg font-bold text-white">CloudCart Admin</span>
        </div>

        <form onSubmit={handleSubmit} className="rounded-card bg-paper p-6 shadow-pop">
          <h1 className="font-display text-lg font-semibold text-ink">Admin sign in</h1>
          <div className="mt-5 space-y-4">
            <div>
              <label className="label">Email</label>
              <input type="email" required className="input" value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} />
            </div>
            <div>
              <label className="label">Password</label>
              <input type="password" required className="input" value={form.password} onChange={(e) => setForm({ ...form, password: e.target.value })} />
            </div>
            {error && <p className="text-sm text-danger">{error}</p>}
            <button type="submit" disabled={submitting} className="btn-primary w-full">
              {submitting ? "Signing in..." : "Sign in"}
            </button>
          </div>
          <p className="mt-4 rounded-lg bg-cargo-light px-3 py-2 text-xs text-cargo">
            Demo: <span className="font-mono">admin@cloudcart.com</span> / <span className="font-mono">Admin@123</span>
          </p>
        </form>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import * as adminApi from "../../api/adminApi";
import AdminTopbar from "../../components/layout/AdminTopbar";
import StatusBadge from "../../components/common/StatusBadge";
import Spinner from "../../components/common/Spinner";

function StatCard({ label, value, sublabel }) {
  return (
    <div className="card p-5">
      <p className="eyebrow">{label}</p>
      <p className="mt-2 font-mono text-2xl font-bold text-ink">{value}</p>
      {sublabel && <p className="mt-1 text-xs text-slate">{sublabel}</p>}
    </div>
  );
}

function RevenueChart({ data }) {
  const max = Math.max(...data.map((d) => d.revenue));
  return (
    <div className="card p-5">
      <h3 className="font-display font-semibold text-ink">Revenue, last 6 months</h3>
      <div className="mt-6 flex h-40 items-end gap-4">
        {data.map((d) => (
          <div key={d.month} className="flex flex-1 flex-col items-center gap-2">
            <div
              className="w-full rounded-t-md bg-cargo transition-all"
              style={{ height: `${(d.revenue / max) * 100}%`, minHeight: 4 }}
              title={` ${d.revenue.toLocaleString()} `}
            </div>
            <span className="font-mono text-xs text-slate">{d.month}</span>
          </div>
        ))}
      </div>
    </div>
  );
}

export default function Dashboard() {
  const [stats, setStats] = useState(null);

  useEffect(() => {
    adminApi.getDashboardStats().then(setStats);
  }, []);

  if (!stats) return <div className="flex justify-center py-24"><Spinner size={32} /></div>;

  return (
    <div>
      <AdminTopbar title="Dashboard" />
      <div className="p-6 space-y-6">
        <div className="grid grid-cols-2 gap-4 lg:grid-cols-5">
          <StatCard label="Revenue" value={` ${stats.totalRevenue.toLocaleString()} `} />
          <StatCard label="Orders" value={stats.totalOrders} />
          <StatCard label="Customers" value={stats.totalCustomers} />
          <StatCard label="Products" value={stats.totalProducts} />
          <StatCard label="Low stock" value={stats.lowStockCount} sublabel="< 10 units" />
        </div>

        <RevenueChart data={stats.revenueByMonth} />

        <div className="card p-5">
          <div className="flex items-center justify-between">
            <h3 className="font-display font-semibold text-ink">Recent orders</h3>
            <Link to="/admin/orders" className="text-sm text-cargo hover:underline">View all </Link>
          </div>
          <table className="mt-4 w-full text-sm">
            <thead>
              <tr className="border-b border-line text-left text-xs uppercase tracking-wide text-slate">
                <th className="pb-2 font-medium">Order</th>
                <th className="pb-2 font-medium">Date</th>
                <th className="pb-2 font-medium">Total</th>
                <th className="pb-2 font-medium">Status</th>
              </tr>
            </thead>
            <tbody>
              {stats.recentOrders.map((o) => (
                <tr key={o.id} className="border-b border-line last:border-0">
                  <td className="py-2.5 font-mono">#{o.id}</td>
                  <td className="py-2.5 text-slate">{o.placedAt}</td>
                  <td className="py-2.5 price">${o.total.toFixed(2)}</td>
                  <td className="py-2.5"><StatusBadge status={o.status} /></td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import * as productApi from "../../api/productApi";
import * as categoryApi from "../../api/categoryApi";
import AdminTopbar from "../../components/layout/AdminTopbar";
import Spinner from "../../components/common/Spinner";

const emptyForm = { name: "", categoryId: "", price: "", stock: "", description: "", image: "product" };

export default function AdminProducts() {
  const [products, setProducts] = useState([]);
  const [categories, setCategories] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);
  const [editingId, setEditingId] = useState(null);
  const [form, setForm] = useState(emptyForm);
  const [saving, setSaving] = useState(false);

  const load = async () => {
    setLoading(true);
    const [prodRes, cats] = await Promise.all([productApi.getProducts({ size: 100 }), categoryApi.getCategories()]);
    setProducts(prodRes.content);
    setCategories(cats);
    setLoading(false);
  };

  useEffect(() => {
    load();
  }, []);

  const openCreate = () => {
    setForm(emptyForm);
    setEditingId(null);
    setShowForm(true);
  };

  const openEdit = (product) => {
    setForm({
      name: product.name,
      categoryId: product.categoryId,
      price: product.price,
      stock: product.stock,
      description: product.description,
      image: product.image,
    });
    setEditingId(product.id);
    setShowForm(true);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setSaving(true);
    const payload = { ...form, categoryId: Number(form.categoryId), price: Number(form.price), stock: Number(form.stock) };
    try {
      if (editingId) await productApi.updateProduct(editingId, payload);
      else await productApi.createProduct(payload);
      setShowForm(false);
      await load();
    } finally {
      setSaving(false);
    }
  };

  const handleDelete = async (id) => {
    if (!confirm("Delete this product? This cannot be undone.")) return;
    await productApi.deleteProduct(id);
    await load();
  };

  return (
    <div>
      <AdminTopbar title="Products" />
      <div className="p-6">
        <div className="mb-4 flex justify-end">
          <button onClick={openCreate} className="btn-primary">+ Add product</button>
        </div>

        <div className="card overflow-x-auto p-5">
          {loading ? (
            <div className="flex justify-center py-12"><Spinner size={28} /></div>
          ) : (
            <table className="w-full min-w-[640px] text-sm">
              <thead>
                <tr className="border-b border-line text-left text-xs uppercase tracking-wide text-slate">
                  <th className="pb-2 font-medium">Product</th>
                  <th className="pb-2 font-medium">Category</th>
                  <th className="pb-2 font-medium">Price</th>
                  <th className="pb-2 font-medium">Stock</th>
                  <th className="pb-2 font-medium text-right">Actions</th>
                </tr>
              </thead>
              <tbody>
                {products.map((p) => (
                  <tr key={p.id} className="border-b border-line last:border-0">
                    <td className="py-2.5">{p.name}</td>
                    <td className="py-2.5 text-slate">{categories.find((c) => c.id === p.categoryId)?.name || ""}</td>
                    <td className="py-2.5 price">${p.price.toFixed(2)}</td>
                    <td className="py-2.5 font-mono">{p.stock}</td>
                    <td className="py-2.5 text-right space-x-2">
                      <button onClick={() => openEdit(p)} className="text-cargo hover:underline">Edit</button>
                      <button onClick={() => handleDelete(p.id)} className="text-danger hover:underline">Delete</button>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
        </div>
      </div>
    </div>
  );
}

```

```

    </div>
  </div>

  {showForm && (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-ink/40 p-4">
      <form onSubmit={handleSubmit} className="w-full max-w-lg rounded-card bg-paper p-6 shadow-pop">
        <h2 className="font-display text-lg font-semibold text-ink">{editingId ? "Edit product" : "Add product"}</h2>
        <div className="mt-4 space-y-3">
          <div>
            <label className="label">Name</label>
            <input required className="input" value={form.name} onChange={(e) => setForm({ ...form, name: e.target.value })} />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div>
              <label className="label">Category</label>
              <select required className="input" value={form.categoryId} onChange={(e) => setForm({ ...form, categoryId: e.target.value })}>
                <option value="">Select...</option>
                {categories.map((c) => <option key={c.id} value={c.id}>{c.name}</option>)}
              </select>
            </div>
            <div>
              <label className="label">Price ($)</label>
              <input required type="number" step="0.01" min="0" className="input" value={form.price} onChange={(e) => setForm({ ...form, price: e.target.value })} />
            </div>
          </div>
          <div>
            <label className="label">Stock quantity</label>
            <input required type="number" min="0" className="input" value={form.stock} onChange={(e) => setForm({ ...form, stock: e.target.value })} />
          </div>
          <div>
            <label className="label">Description</label>
            <textarea rows={3} className="input" value={form.description} onChange={(e) => setForm({ ...form, description: e.target.value })} />
          </div>
        </div>
        <div className="mt-5 flex justify-end gap-3">
          <button type="button" onClick={() => setShowForm(false)} className="btn-ghost">Cancel</button>
          <button type="submit" disabled={saving} className="btn-primary">{saving ? "Saving..." : "Save product"}</button>
        </div>
      </form>
    </div>
  )}
</div>
);
}

```

```

import { useEffect, useState } from "react";
import * as categoryApi from "../../api/categoryApi";
import AdminTopbar from "../../components/layout/AdminTopbar";
import Spinner from "../../components/common/Spinner";

export default function AdminCategories() {
  const [categories, setCategories] = useState([]);
  const [loading, setLoading] = useState(true);
  const [name, setName] = useState("");
  const [editingId, setEditingId] = useState(null);
  const [editingName, setEditingName] = useState("");

  const load = async () => {
    setLoading(true);
    setCategories(await categoryApi.getCategories());
    setLoading(false);
  };

  useEffect(() => {
    load();
  }, []);

  const handleAdd = async (e) => {
    e.preventDefault();
    if (!name.trim()) return;
    await categoryApi.createCategory({ name: name.trim(), slug: name.trim().toLowerCase().replace(/\s+/g, "-") });
    setName("");
    await load();
  };

  const handleSaveEdit = async (id) => {
    await categoryApi.updateCategory(id, { name: editingName });
    setEditingId(null);
    await load();
  };

  const handleDelete = async (id) => {
    if (!confirm("Delete this category? Products in it will need reassigning.)) return;
    await categoryApi.deleteCategory(id);
    await load();
  };

  return (
    <div>
      <AdminTopbar title="Categories" />
      <div className="p-6">
        <form onSubmit={handleAdd} className="card mb-5 flex gap-3 p-4">
          <input
            className="input"
            placeholder="New category name.."
            value={name}
            onChange={(e) => setName(e.target.value)}
          />
          <button type="submit" className="btn-primary shrink-0">+ Add</button>
        </form>

        <div className="card divide-y divide-line p-2">
          {loading ? (
            <div className="flex justify-center py-12"><Spinner size={28} /></div>
          ) : (
            categories.map((cat) => (
              <div key={cat.id} className="flex items-center justify-between px-3 py-3">
                {editingId === cat.id ? (
                  <input className="input mr-3" value={editingName} onChange={(e) => setEditingName(e.target.value)} />
                ) : (
                  <span className="text-sm text-ink">{cat.name}</span>
                )}
                <div className="flex shrink-0 gap-3 text-sm">
                  {editingId === cat.id ? (
                    <button onClick={() => handleSaveEdit(cat.id)} className="text-cargo hover:underline">Save</button>
                  ) : (
                    <button onClick={() => { setEditingId(cat.id); setEditingName(cat.name); }} className="text-cargo hover:underline">Edit</button>
                  )}
                  <button onClick={() => handleDelete(cat.id)} className="text-danger hover:underline">Delete</button>
                </div>
              </div>
            ))
          )}
        </div>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import * as productApi from "../../api/productApi";
import AdminTopbar from "../../components/layout/AdminTopbar";
import Spinner from "../../components/common/Spinner";

export default function AdminInventory() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [filter, setFilter] = useState("ALL");

  const load = async () => {
    setLoading(true);
    const { content } = await productApi.getProducts({ size: 100 });
    setProducts(content);
    setLoading(false);
  };

  useEffect(() => {
    load();
  }, []);

  const adjustStock = async (id, delta) => {
    const product = products.find((p) => p.id === id);
    const newStock = Math.max(0, product.stock + delta);
    await productApi.updateProduct(id, { stock: newStock });
    setProducts((prev) => prev.map((p) => (p.id === id ? { ...p, stock: newStock } : p)));
  };

  const filtered = products.filter((p) => {
    if (filter === "LOW") return p.stock > 0 && p.stock < 10;
    if (filter === "OUT") return p.stock === 0;
    return true;
  });

  return (
    <div>
      <AdminTopbar title="Inventory" />
      <div className="p-6">
        <div className="mb-4 flex gap-2">
          {[
            { key: "ALL", label: "All" },
            { key: "LOW", label: "Low stock" },
            { key: "OUT", label: "Out of stock" },
          ].map((f) => (
            <button
              key={f.key}
              onClick={() => setFilter(f.key)}
              className={`rounded-full px-4 py-1.5 text-sm font-medium ${filter === f.key ? "bg-cargo text-white" : "bg-paper border border-line text-ink hover:bg-mist"}`}
            >
              {f.label}
            </button>
          ))}
        </div>

        <div className="card overflow-x-auto p-5">
          {loading ? (
            <div className="flex justify-center py-12"><Spinner size={28} /></div>
          ) : (
            <table className="w-full min-w-[560px] text-sm">
              <thead>
                <tr className="border-b border-line text-left text-xs uppercase tracking-wide text-slate">
                  <th className="pb-2 font-medium">Product</th>
                  <th className="pb-2 font-medium">Stock</th>
                  <th className="pb-2 font-medium">Status</th>
                  <th className="pb-2 font-medium text-right">Adjust</th>
                </tr>
              </thead>
              <tbody>
                {filtered.map((p) => (
                  <tr key={p.id} className="border-b border-line last:border-0">
                    <td className="py-2.5">{p.name}</td>
                    <td className="py-2.5 font-mono">{p.stock}</td>
                    <td className="py-2.5">
                      {p.stock === 0 ? (
                        <span className="badge-danger">Out of stock</span>
                      ) : p.stock < 10 ? (
                        <span className="badge-warn">Low stock</span>
                      ) : (
                        <span className="badge-ok">Healthy</span>
                      )}
                    </td>
                    <td className="py-2.5 text-right">
                      <div className="inline-flex items-center gap-2">
                        <button onClick={() => adjustStock(p.id, -1)} className="h-7 w-7 rounded-full border border-line hover:bg-mist">-</button>
                        <button onClick={() => adjustStock(p.id, 1)} className="h-7 w-7 rounded-full border border-line hover:bg-mist">+</button>
                      </div>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
        </div>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import * as adminApi from "../../api/adminApi";
import AdminTopbar from "../../components/layout/AdminTopbar";
import StatusBadge from "../../components/common/StatusBadge";
import Spinner from "../../components/common/Spinner";

const STATUSES = ["PLACED", "PROCESSING", "SHIPPED", "DELIVERED", "CANCELLED"];

export default function AdminOrders() {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(true);

  const load = async () => {
    setLoading(true);
    setOrders(await adminApi.getAllOrders());
    setLoading(false);
  };

  useEffect(() => {
    load();
  }, []);

  const handleStatusChange = async (id, status) => {
    await adminApi.updateOrderStatus(id, status);
    setOrders((prev) => prev.map((o) => (o.id === id ? { ...o, status } : o)));
  };

  return (
    <div>
      <AdminTopbar title="Orders" />
      <div className="p-6">
        <div className="card overflow-x-auto p-5">
          {loading ? (
            <div className="flex justify-center py-12"><Spinner size={28} /></div>
          ) : (
            <table className="w-full min-w-[640px] text-sm">
              <thead>
                <tr className="border-b border-line text-left text-xs uppercase tracking-wide text-slate">
                  <th className="pb-2 font-medium">Order</th>
                  <th className="pb-2 font-medium">Date</th>
                  <th className="pb-2 font-medium">Total</th>
                  <th className="pb-2 font-medium">Status</th>
                  <th className="pb-2 font-medium text-right">Update status</th>
                </tr>
              </thead>
              <tbody>
                {orders.map((o) => (
                  <tr key={o.id} className="border-b border-line last:border-0">
                    <td className="py-2.5 font-mono">#{o.id}</td>
                    <td className="py-2.5 text-slate">{o.placedAt}</td>
                    <td className="py-2.5 price">${o.total.toFixed(2)}</td>
                    <td className="py-2.5"><StatusBadge status={o.status} /></td>
                    <td className="py-2.5 text-right">
                      <select
                        className="input !w-auto !py-1.5 text-xs"
                        value={o.status}
                        onChange={(e) => handleStatusChange(o.id, e.target.value)}
                      >
                        {STATUSES.map((s) => <option key={s} value={s}>{s}</option>)}
                      </select>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
        </div>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import * as adminApi from "../../api/adminApi";
import AdminTopbar from "../../components/layout/AdminTopbar";
import Spinner from "../../components/common/Spinner";

export default function AdminCustomers() {
  const [customers, setCustomers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [statusMap, setStatusMap] = useState({});

  useEffect(() => {
    adminApi.getCustomers().then((data) => {
      setCustomers(data);
      setLoading(false);
    });
  }, []);

  const toggleStatus = async (id) => {
    const next = statusMap[id] === "BLOCKED" ? "ACTIVE" : "BLOCKED";
    await adminApi.setCustomerStatus(id, next);
    setStatusMap((prev) => ({ ...prev, [id]: next }));
  };

  return (
    <div>
      <AdminTopbar title="Customers" />
      <div className="p-6">
        <div className="card overflow-x-auto p-5">
          {loading ? (
            <div className="flex justify-center py-12"><Spinner size={28} /></div>
          ) : (
            <table className="w-full min-w-[640px] text-sm">
              <thead>
                <tr className="border-b border-line text-left text-xs uppercase tracking-wide text-slate">
                  <th className="pb-2 font-medium">Name</th>
                  <th className="pb-2 font-medium">Email</th>
                  <th className="pb-2 font-medium">Orders</th>
                  <th className="pb-2 font-medium">Total spent</th>
                  <th className="pb-2 font-medium">Joined</th>
                  <th className="pb-2 font-medium text-right">Action</th>
                </tr>
              </thead>
              <tbody>
                {customers.map((c) => {
                  const status = statusMap[c.id] || "ACTIVE";
                  return (
                    <tr key={c.id} className="border-b border-line last:border-0">
                      <td className="py-2.5">{c.name}</td>
                      <td className="py-2.5 text-slate">{c.email}</td>
                      <td className="py-2.5 font-mono">{c.orders}</td>
                      <td className="py-2.5 price">${c.totalSpent.toFixed(2)}</td>
                      <td className="py-2.5 text-slate">{c.joined}</td>
                      <td className="py-2.5 text-right">
                        <button
                          onClick={() => toggleStatus(c.id)}
                          className={status === "BLOCKED" ? "text-ok hover:underline" : "text-danger hover:underline"}
                        >
                          {status === "BLOCKED" ? "Unblock" : "Block"}
                        </button>
                      </td>
                    </tr>
                  );
                })}
              </tbody>
            </table>
          )}
        </div>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from "react";
import * as adminApi from "../../api/adminApi";
import AdminTopbar from "../../components/layout/AdminTopbar";
import Spinner from "../../components/common/Spinner";

export default function AdminReports() {
  const [report, setReport] = useState(null);

  useEffect(() => {
    adminApi.getSalesReport().then(setReport);
  }, []);

  if (!report) return <div className="flex justify-center py-24"><Spinner size={32} /></div>;

  return (
    <div>
      <AdminTopbar title="Reports" />
      <div className="p-6 space-y-6">
        <div className="grid grid-cols-1 gap-4 sm:grid-cols-3">
          <div className="card p-5">
            <p className="eyebrow">Total revenue</p>
            <p className="mt-2 font-mono text-2xl font-bold text-ink">${report.totalRevenue.toLocaleString()}</p>
          </div>
          <div className="card p-5">
            <p className="eyebrow">Total orders</p>
            <p className="mt-2 font-mono text-2xl font-bold text-ink">{report.totalOrders}</p>
          </div>
          <div className="card p-5">
            <p className="eyebrow">Avg. order value</p>
            <p className="mt-2 font-mono text-2xl font-bold text-ink">${report.averageOrderValue.toFixed(2)}</p>
          </div>
        </div>

        <div className="card p-5">
          <h3 className="font-display font-semibold text-ink">Top selling products</h3>
          <p className="text-xs text-slate">{report.from} – {report.to}</p>
          <table className="mt-4 w-full text-sm">
            <thead>
              <tr className="border-b border-line text-left text-xs uppercase tracking-wide text-slate">
                <th className="pb-2 font-medium">Product</th>
                <th className="pb-2 font-medium">Units sold</th>
                <th className="pb-2 font-medium">Revenue</th>
              </tr>
            </thead>
            <tbody>
              {report.topProducts.map((p) => (
                <tr key={p.name} className="border-b border-line last:border-0">
                  <td className="py-2.5">{p.name}</td>
                  <td className="py-2.5 font-mono">{p.unitsSold}</td>
                  <td className="py-2.5 price">${p.revenue.toLocaleString()}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
}

```

```
# CloudCart – Frontend

React + Vite storefront and admin console for the CloudCart online shopping portal.

## Stack

- React 18, React Router 6
- Tailwind CSS (custom CloudCart design tokens – see `tailwind.config.js`)
- Axios with JWT access/refresh token interceptors
- Context API for auth and cart state (no Redux needed at this scale)

## Getting started

```bash
npm install
cp .env.example .env
npm run dev
```

The app opens at `http://localhost:5173`.

By default `VITE_USE MOCK=true`, so the whole app – products, cart, orders, reviews, wishlist, admin dashboard – runs on realistic in-memory sample data with no backend required. This lets you review every screen immediately.

## Connecting to the real backend

1. Start the Spring Boot backend (see `../cloudcart-backend`).
2. In `.env`, set:
   ```
 VITE_API_BASE_URL=http://localhost:8080/api
 VITE_USE MOCK=false
   ```
3. Restart `npm run dev`. Every `src/api/*.js` file already calls the exact REST endpoints documented in the backend's API docs – no code changes needed, just the flag.

## Demo accounts (mock mode)



Role	Email	Password
Customer	asha@example.com	Password@123
Admin	admin@cloudcart.com	Admin@123



## Project structure

```
src/
 api/ REST service modules (one per resource) + mock data
 context/ AuthContext, CartContext
 components/
 layout/ Navbar, Footer, Admin sidebar/topbar, layout shells
 common/ Buttons, spinners, route guards, pagination, empty states
 product/ ProductCard, ProductGrid, CategoryFilter, reviews
 cart/ Cart row, order summary
 pages/
 customer/ Home, Login, Register, Products, ProductDetails, Cart,
 Checkout, Orders, OrderDetails, Wishlist, Profile
 admin/ AdminLogin, Dashboard, Products, Categories, Inventory,
 Orders, Customers, Reports
```

## Design system

Palette, type, and layout tokens are documented as comments at the top of `tailwind.config.js`. In short: indigo (`cargo`) as the primary brand color, warm coral (`crate`) as the call-to-action accent, Space Grotesk for headings, Inter for body text, and IBM Plex Mono for prices, SKUs, and admin data – giving the admin console a "control room" feel distinct from the friendlier storefront.

## Build

```bash
npm run build
```

Outputs a static bundle to `dist/`, ready to serve from S3 + CloudFront, an Nginx container, or any static host as described in the project's AWS deployment guide.
```